



1.	INSTALACIÓN	3
2.	CREACIÓN DE UN NUEVO PROYECTO	6
	Configuración del Proyecto	10
	Levantar el servidor de desarrollo	12
3.	PRIMEROS PASOS	14
	Desde el router	14
	Desde un controlador.....	15
	Desde las vistas.....	17
4.	MVC EN LARAVEL.....	22
5.	MODELO.....	23
	Migraciones	24
	Seeders (sembradores).....	27
	Modelos.....	29
6.	CONTROLADOR.....	37
	Crear Controlador	37
7.	CONFIGURAR LAS RUTAS.....	39
8.	VISTAS DINÁMICAS	41
9.	GESTIÓN DE PARÁMETROS EN LA RUTA Y DATOS EN BODY (POST) CON VALIDACIONES	52
10.	CREACIÓN DE UNA API REST	66
11.	LARAVEL BREEZE	71
12.	AUTENTICACIÓN DE APIs CON SANCTUM	72
13.	INCLUIR BREEZE BLADE PARA AUTENTICACIÓN DE USUARIOS EN WEB TRADICIONAL.....	76
14.	APLICACIÓN PRIETO EATS.....	78
15.	GESTIÓN DE SESIONES	84
	Dónde se almacenan las sesiones en Laravel	84
	Usos principales de las sesiones en una aplicación web.....	84
	Cómo usar sesiones en Laravel	86
16.	GESTIÓN DEL CARRITO DE LA COMPRA.....	88
	Uso de la sesión para el carrito (basado en sesiones, sin persistencia).....	89
	Uso del carrito con persistencia en bases de datos	94
17.	GESTIÓN DE PERFILES/ROLES DE USUARIOS	98
	Opción 1: Agregar una columna string "role" o boolean "is_admin" en la tabla users	98
	Opción 2: Crear tablas roles y user_role	102
	Opción 3: Instalar la librería Spatie Laravel Permission	102
18.	PRIETO EATS. CREACIÓN DE NUEVA OFERTA CON CHECKBOX PARA ELEGIR PRODUCTOS.....	103
19.	PRIETO EATS. MODIFICACIÓN DEL CARRITO DE LA COMPRA PRIETO EATS A OFERTAS	105
20.	VALIDACIÓN EN SERVIDOR Y EN CLIENTE DE FORMULARIOS.....	109

Laravel es un framework de código abierto creado en el 2011 para el desarrollo de aplicaciones a la medida con PHP, que pone énfasis en el desarrollo de código de forma organizada y sencilla para facilitar las tareas de mantenibilidad y escalabilidad de los desarrollos.

Ventajas:

- Patrón MVC (Modelo Vista Controlador)
- Sistema de plantillas conocido como Blade para construir las Interfaces Gráficas de Usuario (GUI)
- Compatible con la mayoría de servidores de bases de datos relacionales
- Incorpora ORM Eloquent para interactuar con bases de datos relacionales utilizando objetos en lugar de escribir sentencias SQL directas.
- Cuenta con características de seguridad preconstruida que previenen los ataques mediante inyecciones SQL y los ataques Cross Site Request Forgery (CSRF)
- Soporte a largo plazo (LTS)
- Comunidad activa

1. INSTALACIÓN

Instalar PHP → instalar XAMPP

Si ya has instalado XAMPP, que incorpora PHP, es posible instalar otra versión de PHP distinta a la que trae XAMPP, para ello:

- Descarga la versión que quieras de PHP desde <https://www.php.net/> (yo tengo ahora la versión 8.2.27)
- Descarga la versión apropiada:
 - Para desarrollo local, elige la versión **Thread Safe** (requerida para integrarse con Apache).
 - Elige la versión Thread Safe (TS) o Non Thread Safe (NTS) según sea el servidor web que utilizas. En el caso de Apache, que maneja múltiples hilos (threads) simultáneamente, elige la versión TS.
 - En el caso de usar el servidor embebido de PHP, entonces debes instalar la versión Non Thread Safe (NTS).
 - Descarga el archivo **ZIP** correspondiente a tu sistema operativo (x64 para 64 bits).
- Extrae el archivo ZIP descargado en un directorio
- Renombra la carpeta xampp/php a xampp/phpBackup, para no escribirlo encima y si no funciona podamos volver a ponerla.
- Crea la carpeta xampp/php y copia los archivos descargados de php con la versión deseada.
- Copia el archivo php.ini de /xampp/phpBackup a /xampp/php
- Cierra la terminal y ábrela de nuevo
- Comprueba la versión de php que está funcionando → `php -version`

Instalar Composer

Laravel requiere Composer para gestionar sus dependencias, esto es, el conjunto de ficheros de las librerías que necesita.

- **Descargar instalador composer**

<https://getcomposer.org/download>

- **Ejecutar este instalador**

☒ **Developer mode**
Take control and just install Composer. An uninstaller will not be included.

Select Destination Location
Where should Composer be installed?
 Setup will install Composer into the following folder.
To continue, click Next. If you would like to select a different folder, click Browse.

Settings Check
We need to check your PHP and other settings.
Choose the command-line PHP you want to use:

☒ Add this PHP to your path?

- **No proxy**

Asegúrate de marcar la opción para agregar Composer al **PATH**

- **Verificar la instalación:**

```
C:\Users\Paula>composer -v
Composer
Composer version 2.7.7 2024-06-10 22:11:12
Usage:
  command [options] [arguments]
```

- **Configuración adicional (opcional)**

Edita el archivo php.ini y descomenta las líneas:

```
extension=openssl  
extension=curl  
extension=mbstring
```

Instalar Laravel installer (no es obligatorio)

Laravel Installer es una herramienta que facilita la creación de nuevos proyectos Laravel desde la línea de comandos.

Instalar Laravel installer globalmente:

```
composer global require laravel/installer
```

Podrás ver la versión instalada con el comando:

```
laravel --version
```

Se instala la última versión compatible según tu entorno PHP. Si tenías PHP 8.2.27, se instalará Laravel 5.11.2.

Si deseas actualizar el instalador a la versión más reciente, utiliza:

```
composer global update laravel/installer
```

Si no actualiza y prefieres borrar Laravel installer y volver a instalarlo:

```
composer global remove laravel/installer
```

A Nov de 2025 la versión de Laravel installer es:

```
Using version ^5.23 for laravel/installer
```

Manejo de ficheros zip

Para que Laravel installer y Composer no tengan problemas para descargar algunos paquetes de los proyectos desde los archivos zip preempaquetados y no tengan que descargarlos desde el código fuente en el repositorio de GitHub o similar, es conveniente realizar lo siguiente:

- Habilitar la extensión zip en PHP
Abre el fichero php.ini y descomenta o escribe la línea:

```
extensión=zip
```


Guárdalo y reinicia la terminal para que los cambios surtan efecto.
Puedes comprobar que está habilitada esta extensión con → `php -m | findstr zip`
- Instala algún compresor para esta extensión: [unzip](#) o [7z](#)

Habilitar la extensión fileinfo en php.ini

```
extension=fileinfo
```

Es una extensión de PHP necesaria para que Laravel pueda trabajar con archivos (subidas, tipos MIME, etc.).

2. CREACIÓN DE UN NUEVO PROYECTO

Es posible instalar un nuevo proyecto con Composer directamente:

```
composer create-project laravel/laravel 01Proyecto
```

Descarga Laravel base o mínimo, sin ningún asistente que realice preguntas respecto a si se instalan o no ciertos starter kits, o conjunto de código ya preparado que Laravel puede añadir a un proyecto nuevo para darle funcionalidades básicas listas para usar, sin que haya que programarlas desde cero. Se puede añadir un starter kit en cualquier momento.

Otro modo, si has instalado Laravel installer:

laravel new nombre-proyecto

Laravel installer tiene un asistente interactivo que pregunta por la instalación de starter kits.

Vamos a utilizar composer ya que nos crea el proyecto base para empezar a trabajar, conforme vayamos necesitando nuevas funcionalidades, las iremos instalando "a mano".

Se instalará una versión de proyecto Laravel con el que se va a trabajar. Para saber cuál es esta versión:

php artisan --version

Lo que devolverá, por ejemplo:

```
Laravel Framework 12.39.0
```

Es posible **instalar una versión concreta del framework Laravel al crear un proyecto nuevo.**

laravel new nombre-proyecto --version="10.0"

Esto creará un nuevo proyecto con Laravel **10.0** exactamente (si está disponible esa versión).

Si, en cambio, no usas el instalador global o prefieres Composer:

```
composer create-project laravel/laravel nombre-proyecto "10.0.*"
```

Si quieres cambiar la versión de un proyecto existente:

- Editas el archivo composer.json
- Buscas la línea → "laravel/framework": "^10.0",
- Cambias el número por la versión deseada
- Ejecutas → composer update

Precaución: Cambiar de versión puede romper cosas si subes o bajas mucho de versión, así que siempre revisa la [guía de actualización oficial](#) para evitar problemas.



Programación Web en entorno servidor **LARAVEL**



Según sea la versión de Laravel, el comando “**laravel new nombreProyecto**” puede realizar algunas preguntas para configurar el proyecto según tus necesidades:

- **Would you like to install a starter kit?**

Esta pregunta aparece para ofrecerte opciones de starter kits que Laravel incluye una plantilla base para comenzar tu proyecto, con autenticación, frontend y estructura básica.

None:

No instala ningún starter kit.

El proyecto será básico, con solo los archivos mínimos de Laravel. Ideal si quieres una API REST pura, sin frontend.

No incluye rutas de autenticación, controladores de autenticación, ni vistas de login/register, sin migraciones para usuarios o sesiones, sin CSS ni frontend montado.

react:

Instala Laravel Breeze + Inertia.js + React.

Incluye autenticación básica (login, registro, logout) --> Breeze

Usa React en el frontend.

Inertia.js un adaptador que permite construir aplicaciones SPA (Single Page Application) usando frameworks modernos de JavaScript (como Vue, React o Svelte) junto con un backend tradicional en Laravel, sin necesidad de crear una API REST completa.

Necesitarás instalar dependencias con npm install y compilar con npm run dev.

vue:

Igual que la anterior pero con Vue.js en lugar de React.

También usa Inertia.js.

Livewire:

Usa Laravel Breeze + Livewire + Blade.

Recomendado si prefieres trabajar con Blade + PHP sin JavaScript avanzado.

Livewire te permite crear componentes interactivos (como formularios dinámicos, filtros, tablas reactivas, etc.) directamente en Laravel, y se encarga de sincronizar los cambios entre el frontend y el backend vía AJAX en segundo plano.

Muy útil para apps tradicionales con frontend Laravel.

- **Which testing framework do you prefer?**

Pest: es un framework moderno para pruebas en PHP, construido sobre PHPUnit.

PHPUnit: es el framework de testing tradicional y oficial para PHP.

- **Do you want to install Laravel Boost to improve AI assisted coding?**

Laravel Boost es una nueva herramienta desarrollada por el equipo de Laravel para mejorar la asistencia de IA en el desarrollo. Su objetivo principal es:

- Proveer archivos enriquecidos y metadatos que permiten que herramientas como Laravel AI, GitHub Copilot, o entornos con IA (como ChatGPT) comprendan mejor tu aplicación.

- No es obligatorio ni cambia el comportamiento de tu app.

- **Which database will your application use?**

```
Which database will your application use? [MySQL]:  
[mysql ] MySQL  
[mariadb] MariaDB  
[sqlite ] SQLite (Missing PDO extension)  
[pgsql ] PostgreSQL (Missing PDO extension)  
[sqlsrv ] SQL Server (Missing PDO extension)
```

- **Default database updated. Would you like to run the default database migrations?**

Si indicas que sí, debes tener el motor de la Base de Datos ejecutándose, ya que se crearán las migraciones (tablas) iniciales que traiga el proyecto por defecto.

Si el proyecto incluye migraciones para configurar la base de datos, Laravel pregunta si deseas ejecutarlas de inmediato.

Yes: Ejecuta las migraciones automáticamente con **php artisan migrate**

No: No ejecuta las migraciones automáticamente. Deberás hacerlo más tarde

- **Would you like to run npm install and npm run build?**

Laravel te está preguntando si quieres que se ejecuten automáticamente los siguientes comandos:

1. **npm install**

- Descarga todas las dependencias JavaScript definidas en package.json.
- Incluye cosas como Vite, TailwindCSS, Vue/React, etc. (según el starter kit que elegiste).

2. **npm run build**

- Compila los archivos del frontend para producción (JavaScript, CSS...).
- Usa Vite como herramienta de compilación.

En el caso de elegir Livewire, nos realiza las siguientes preguntas:

```
Which authentication provider do you prefer? [Laravel's built-in authentication]:  
[laravel] Laravel's built-in authentication  
[workos ] WorkOS (Requires WorkOS account)  
[none ] No authentication scaffolding
```

Laravel:

- Instala la autenticación integrada de Laravel (por ejemplo, Breeze, Fortify, o Laravel UI según el caso).
- Ideal si necesitas registro, login, logout, recuperación de contraseña, etc.
- Utiliza Sanctum (en Breeze API) o sesiones (en versiones con vistas).

WorkOS:

- Integra Laravel con WorkOS, una plataforma de autenticación empresarial.
- Requiere una cuenta de WorkOS.
- Se usa si necesitas SSO empresarial (Google Workspace, Microsoft Entra ID, Okta, etc.).
- Útil en entornos corporativos o multi-tenant.

None:

- No se instalará ningún sistema de autenticación.
- Tú debes configurar login, registro, tokens, etc. manualmente.
- Útil si estás construyendo una API sin usuarios autenticados o si quieres control total.

```
Would you like to use Laravel Volt? (yes/no) [yes]:
```

Laravel Volt es una forma moderna de usar Livewire que te permite escribir toda la lógica y la vista de un componente en un solo archivo, ofrece:

- Componentes frontend modernos.
- Sin necesidad de escribir mucho JavaScript.
- Estilo más organizado y reactivo.
- Experiencia tipo “SPA” (Aplicación de Página Única).

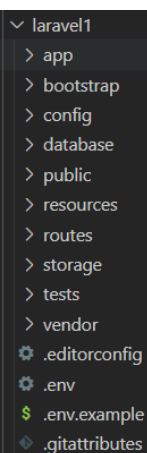
Sí, si quieres usar una forma moderna y rápida de desarrollar interfaces interactivas sin complicarte con JavaScript.

No, si prefieres usar Blade tradicional o manejar tú mismo el frontend (con React, Vue o HTML puro).

Configuración del Proyecto

Se crea un directorio de carpetas donde tendremos el framework instalado:

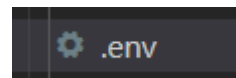
- app: lógica ppal de la aplicación
 - Http: controladores
 - Models: modelos
- bootstrap
- config
- database: contiene las migraciones para crear nuevas tablas y realizar operaciones sobre la BBDD. Contiene los seeders (semillas) que nos ayudarán a introducir información en las tablas
- public: archivos accesibles desde la web (imágenes, css,...)
- resources: contiene los estilos css y las vistas
- routes: contiene los redireccionamientos hacia la lógica del controlador que queramos
- storage: almacena archivos grandes o logs creados dinámicamente
- tests: tests unitarios
- vendor: no se tocará, aquí se van instalando las dependencias



- .env: almacena las variables de entorno de tu aplicación, es decir, todos aquellos valores que pueden cambiar según el entorno donde estés ejecutando el proyecto (local, desarrollo, producción, etc.) sin necesidad de tocar el código.
- composer.json: permite ver las versiones de los paquetes de laravel
- package.json: este archivo es de Node.js y lo usa npm (o Yarn si prefieres) para manejar las dependencias de JavaScript y front-end dentro de Laravel. Está en un proyecto Laravel porque Laravel, aunque es backend PHP, incluye herramientas para trabajar con recursos del front (CSS, JS, etc.) gracias a Laravel Mix o Vite (en las versiones más recientes).

Entra en la carpeta del proyecto.

Configuración inicial BD en el fichero .env



Copia el archivo de entorno base si no existiera:

```
cp .env.example .env
```

Edita las configuraciones básicas en el archivo .env, como conexión a base de datos:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=1_Hola_MundoLaravel
DB_USERNAME=root
DB_PASSWORD=
```

Nombre de la aplicación (se mostrará en el title)

Laravel lo asigna inicialmente en el fichero .env `APP_NAME="Product Store"`, pero en el caso de no estar definido en este fichero, se puede definir en /bootstrap/app.php

```
'name' => env('APP_NAME', 'Laravel'),
```

y posteriormente en la vista que utiliza como plantilla `resources/views/layouts/app.blade.php`

```
<title>{{ config('app.name', 'Laravel') }}</title>
```

Genera la clave de la aplicación (se crea automáticamente)

Garantiza que tu aplicación Laravel sea segura y esté preparada para manejar datos sensibles de manera confiable. Al crear un proyecto nuevo, ésta se crea automáticamente, pero podrías volver a generarla con:

```
php artisan key:generate
```

La clave se almacena en el archivo .env bajo la variable APP_KEY

```
APP_KEY=base64:clave-generada
```

Laravel usa esta clave para encriptar datos confidenciales en tu aplicación. Por ejemplo:

- Tokens de sesión.
- Cookies encriptadas.
- Cualquier dato que necesite almacenamiento seguro en la base de datos.

Ejecuta las migraciones

Si en la instalación no tenías arrancado (ni configurado) el gestor de BBDD, no se habrá creado la Base de Datos y las tablas iniciales. Ejecuta el comando:

```
php artisan migrate
```

Crearé la base de datos y las tablas iniciales (users, sessions,...) aunque realmente no se necesiten aún.

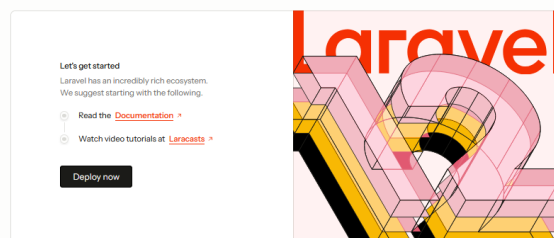
En la tabla migrations se crearán tantos registros como migraciones se hayan ejecutado ya. Si miras en la ruta database/migrations, verás tantos ficheros como registros hay inicialmente en la tabla migrations, cada registro de la tabla se refiere a que se ha ejecutado uno de estos ficheros, es por ello, que si ejecutas de nuevo la instrucción php artisan migrate, dirá que no hay migraciones pendientes, puesto que las que hay, ya se han realizado.

Levantar el servidor de desarrollo

Laravel incluye un servidor web local. Levántalo con:

```
php artisan serve
```

Accede al proyecto en tu navegador en <http://localhost:8000>



Esta pantalla inicial puede variar según sea la versión del framework Laravel instalada en el proyecto

La secuencia de ejecución es:

1. Cuando el cliente hace una petición web (<http://localhost:8000>), se dirige dicha petición a la parte pública de la aplicación, al archivo **public/index.php**, que es el punto de entrada de todas las peticiones en un proyecto Laravel.

Este archivo también gestiona el autoload de Composer (require __DIR__.'../vendor/autoload.php').

2. Este fichero llama a **/bootstrap/app.php**

```
return Application::configure(...)
->withRouting(...)
->withMiddleware(...)
```

```
->create();
```

- Configura e inicializa la instancia de la aplicación.
- Decide internamente qué tipo de rutas ejecutar web (http) o como una consola (console).
- Puedes elegir qué archivos de rutas cargar:

```
->withRouting(  
    api: __DIR__.'/../routes/api.php',  
    web: null, // Puedes no cargarlo  
)
```

3. /routes/web.php, /routes/api.php

La solicitud es manejada por el enrutador de Laravel, que busca una coincidencia con las rutas definidas en los archivos:

- routes/web.php: Rutas para aplicaciones web, suelen devolver vistas.
- routes/api.php: Rutas para APIs, suelen devolver JSON.

Si se encuentra una coincidencia, se ejecuta el controlador o la acción asociada a esa ruta.

4. Controlador de la ruta (aunque no es controlador, es router)

Si la ruta coincide, Laravel ejecuta el controlador o la función Closure (anónima) definida.

```
Route::get('/', function () {  
    return view('welcome');  
});
```

- Si es una función Closure, devuelve directamente el resultado.
- Si es un controlador, Laravel llama al método correspondiente.

5. Renderizar vistas Blade. En el caso de un Proyecto base de Laravel, se llama directamente a una vista /resources/views/welcome.blade.php

Laravel utiliza el motor de plantillas Blade para renderizar vistas, lo que permite usar sintaxis Blade (como directivas @if, @foreach, etc.).

6. Si se llama al controlador, éste puede:

- Consultar la base de datos: Usar modelos de Eloquent o consultas directas.
- Usar servicios o dependencias: Proporcionados por el contenedor de servicios.
- Devolver una respuesta al cliente: Una vista (Blade), JSON, o cualquier otro tipo de respuesta.
- Laravel devuelve la respuesta generada al servidor web (Apache/Nginx o servidor embebido), que se encarga de enviarla al cliente (navegador o cliente HTTP).

3. PRIMEROS PASOS

Desde el router

Devolver texto directamente al cliente

```
Route::get('/hola', function () {  
    return 'Hola mundo';  
});
```

Usar una ruta que sirva el fichero estático:

Si queremos crear una nueva ruta que llame a un fichero estático:

- Crear la ruta nueva en routes/web.php

```
Route::get('/estatico', function () {  
    return response()->file(public_path('static.html'));  
});
```

- Crear en /public/ el fichero static.html
- Ahora puedes acceder al archivo desde: <http://localhost:8000/estatico>

Redirigir directamente a un archivo

- Coloca el archivo estático en /public
- Redirecciona a él desde web.php:

```
Route::get('/estatico', function () {  
    return redirect('static.html');  
});
```

Renderizar una vista de Blade

- Mueve el archivo HTML a la carpeta resources/views/ con el nombre static.blade.php
- Cárgalo como una vista desde /routes/web.php:

```
Route::get('/estatico', function () {  
    return view('static');  
});
```

El hecho de que sea una vista renderizable indica que puede tener código que se ejecutará antes de que la respuesta se envíe al cliente, permitiendo meter datos en la vista o variar el HTML de respuesta al cliente.

En esta ocasión, por simplicidad, renderizamos la vista sin pasarle datos.

Devolver un JSON

```
Route::get('/api/datos', function () {  
    return response()->json(['nombre' => 'Laravel']);  
});
```

Redirigir a otra ruta

```
Route::get('/ir', function () {  
    return redirect('/bienvenida');  
});
```

Llamar a un método de un controlador (que crearemos más tarde)

```
Route::get('/mostrar-datos', [MiController::class, 'mostrar']);
```

Llamará al método mostrar de MiController (lo creamos en el apartado siguiente)

Pero para poder utilizar el controlador, el router ha de importar la clase del controlador (MiController) → **use app\Http\Controllers\MiController;**

Desde un controlador

Creamos un nuevo controlador

```
php artisan make:controller MiController.php
```

Crearé un nuevo controlador en app\Http\Controllers\

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
  
class MiController extends Controller  
{  
    //  
}
```

Creamos un nuevo método

Creamos el método mostrar() que leerá un archivo de la ruta /storage utilizando el método storage_path() que dirige a esa ruta:

```
$texto = file_get_contents(storage_path('app/datos.txt'))
```

Nota: La carpeta storage/ sirve para guardar archivos y datos generados o utilizados por la aplicación. Está pensada para almacenamiento interno, no público

Mandar respuesta al cliente

Llamando (renderizando) una vista, mandándole o no datos como array asociativo

Finalmente, el controlador renderiza una vista (/resources/views/mostrar.blade.php) y le manda los datos leídos del fichero, como un array asociativo:

```
return view('mostrar', ['contenido' => $texto]);
```

Devolviendo texto

```
return 'Respuesta de texto al cliente';
```

Devolviendo JSON

```
return response()->json(['id' => $id, 'nombre' => 'Usuario']);
```

Redireccionando a otra ruta

```
return redirect()->route('otraruta');
```

Volviendo a la página anterior

```
return back();
```

Devolviendo un error 404

```
abort(404)
```

Gestión de peticiones POST

Cuando se recibe una petición POST a una ruta (por ejemplo, desde un formulario), Laravel:

1. Llama al método correspondiente del controlador
2. Inyecta automáticamente una instancia de Illuminate\Http\Request en el parámetro \$request. Por ello se debe importar (use)

Es decir, Laravel no usa directamente \$_POST, sino que lo encapsula en un objeto más potente.

```
public function store(Request $request) {
```


Para obtener datos del formulario en Laravel, tienes varias alternativas:

- `$request->input('campo')`
- `$request->all()` → Devuelve todos los campos del formulario en un array asociativo
- `$request->get('campo')`
- `$request->post('campo')`
- `$request->has('campo')` → Verifica si el campo fue enviado
- `$request->only('a', 'b')` → Solo devuelve los campos indicados

Desde las vistas

Creamos un fichero (vista) en `/resources/views`, que recoja los datos recibidos del controlador con la clave 'contenido'

```
<!DOCTYPE html>
<html>
<head>
  <title>Mostrar Datos</title>
</head>
<body>
  <h1>Contenido del archivo:</h1>
  <pre>{{ $contenido }}</pre>
</body>
</html>
```

Ejercicio:

Realiza un CRUD de usuarios, cuyos datos están almacenados en un fichero usuarios.json (en /storage/app/private).

```
[
  {
    "nombre": "Juan Perez",
    "email": "juan@mail.com",
    "id": 1
  }
]
```

Las rutas serán:

Método	Ruta	Acción del controlador	Descripción
GET	/index	index()	Mostrar todos los usuarios
GET	/create	create()	Formulario para crear nuevo
POST	/store	store()	Guardar el nuevo usuario

Desde el formulario donde se realiza el POST tendremos el action:

`action="{{ route('/store') }}"`

NOTA: Recuerda poner el @csrf (token aleatorio para evitar ataques Cross-Site Request Forgery), lo que creará un control oculto en el formulario con un token aleatorio que será devuelto en el envío del formulario.

NOTA: json_decode() en PHP puede devolver distintos tipos de datos, dependiendo de cómo lo llames:

- json_decode(\$json, false) → devuelve OBJETOS
 - Los objetos JSON se convierten en stdClass
 - Los arrays JSON se convierten en arrays de objetos stdClass

\$datos = json_decode(\$json);

Devuelve:

```
[
  { "id": 1, "nombre": "Ana" },
  { "id": 2, "nombre": "Luis" }
]
```

- json_decode(\$json, true) → devuelve ARRAYS asociativos

```
[
  ["id" => 1, "nombre" => "Ana"],
  ["id" => 2, "nombre" => "Luis"]
]
```

Mejoras del ejercicio anterior:

1) Uso de la facade Storage en la gestión de archivos

Para operar con este fichero puedes utilizar las funciones que conocemos de acceso a ficheros, pero también podemos utilizar la clase Illuminate\Support\Facades\Storage, una fachada (facade) que da acceso al sistema de archivos y almacenamiento de Laravel, permitiendo trabajar con archivos de forma sencilla, ya sea:

- En el disco local (storage/app o en donde esté configurado en config/filesystems.php)
- En Amazon S3
- En Dropbox, FTP, etc. (si lo configuras)

Dispone de las operaciones:

```
$contenido = Storage::get('archivo.txt'); //Leer un archivo
Storage::put('ejemplo.txt', 'Texto de prueba'); //Escribir un archivo. Sobrescribe el archivo si existe
if (Storage::exists('archivo.txt')) { ... } //Verificar si existe
Storage::delete('archivo.txt'); //Eliminar un archivo
Storage::append('ejemplo.txt', 'Línea adicional');
```

2) Uso de nombres en las rutas

En el router (web.php) es posible ponerle nombre a las rutas:

```
Route::get('/index', [UsuarioController::class, 'index'])->name('usuarios.index');
```

Esta opción ofrece la ventaja principal de que se adapta mejor si la URL cambia en el futuro, ya que solo es necesario cambiar la ruta en web.php. Además de que el nombre indica claramente qué hace la ruta (usuarios.show, productos.store, etc.).

3) Ampliación de las rutas del ejercicio

Estas rutas se corresponden con las operaciones CRUD típicas:

Método HTTP	Ruta	Acción del controlador	Nombre de la ruta	Descripción
GET	/usuarios	index()	usuarios.index	Mostrar todos los usuarios
GET	/usuarios/create	create()	usuarios.create	Formulario para crear nuevo
POST	/usuarios	store()	usuarios.store	Guardar el nuevo usuario
GET	/usuarios/{id}	show(\$id)	usuarios.show	Mostrar un usuario concreto
GET	/usuarios/{id}/edit	edit(\$id)	usuarios.edit	Formulario para editar
PUT/PATCH	/usuarios/{id}	update(\$id)	usuarios.update	Actualizar el usuario
DELETE	/usuarios/{id}	destroy(\$id)	usuarios.destroy	Eliminar el usuario

```
Route::get('/usuarios', [UsuarioController::class, 'index'])->name('usuarios.index');
Route::get('/usuarios/create', [UsuarioController::class, 'create'])->name('usuarios.create');
Route::post('/usuarios', [UsuarioController::class, 'store'])->name('usuarios.store');
Route::get('/usuarios/{id}', [UsuarioController::class, 'show'])->name('usuarios.show');
Route::get('/usuarios/{id}/edit', [UsuarioController::class, 'edit'])->name('usuarios.edit');
Route::put('/usuarios/{id}', [UsuarioController::class, 'update'])->name('usuarios.update');
Route::delete('/usuarios/{id}', [UsuarioController::class, 'destroy'])->name('usuarios.destroy');
```

O mejor aún y más breve:

```
Route::resource('usuarios', UsuarioController::class);
```

Laravel crea automáticamente estas 7 rutas RESTful que se corresponden con las operaciones CRUD típicas.

4) Paso de parámetros en las rutas

Ten en cuenta que cuando defines una ruta como:

```
Route::get('/usuarios/{id}', [UsuarioController::class, 'show'])->name('usuarios.show');
```

Laravel interpreta automáticamente cualquier URL que coincida con el patrón `/usuarios/{id}` — por ejemplo `/usuarios/10` — como una coincidencia válida para esa ruta.

- Laravel detecta que `{id}` es un parámetro dinámico.
- En el ejemplo `/usuarios/10`, el número 10 se pasa como argumento al método `show($id)` del controlador.

Incluso puedes aplicar validaciones o restricciones con expresiones regulares:

```
Route::get('/usuarios/{id}', [UsuarioController::class, 'show'])->where('id', '[0-9]+');
```

Desde la vista del cliente llamamos a la ruta edit pasándole el parámetro del id de usuario:

```
<a href="{{ route('usuarios.edit', $usuario['id']) }}">Ver</a>
```

Y desde el formulario de edición se lanzaría un submit con el método PUT y el parámetro con el id del usuario:

```
<form method="POST" action="{{ route('usuarios.update', $usuario['id']) }}">
    @csrf @method('PUT')
    Nombre: <input name="nombre" value="{{ $usuario['nombre'] }}"><br>
    Email: <input name="email" value="{{ $usuario['email'] }}"><br>
    <button type="submit">Actualizar</button>
</form>
```

Aviso: en Laravel es necesario poner en los formularios `@csrf` para evitar estos ataques

5) Uso de otros métodos desde una vista con formulario

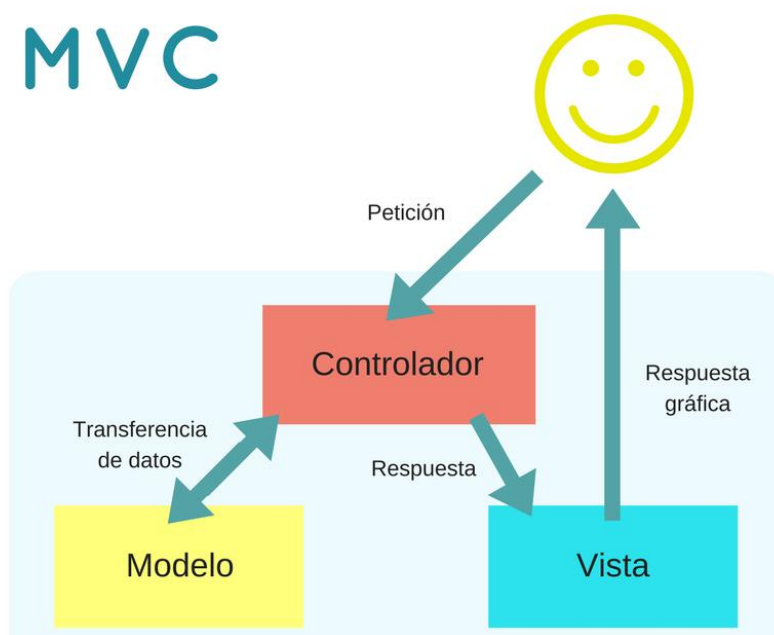
Desde cualquier formulario podemos definir cualquier método, no solamente GET y POST dentro del method, bastará con poner `@method('DELETE')` o bien `@method('PUT')` dentro del formulario, a pesar de que `method = "POST"`.

Ejercicio: Completa el ejercicio anterior con todas las rutas indicadas.

4. MVC EN LARAVEL

El **MVC** es un patrón de diseño arquitectónico de software, que sirve para clasificar la información, la lógica del sistema y la interfaz que se le presenta al usuario. En este tipo de arquitectura existe:

1. **Controlador (Controller)**: un sistema central o controlador que gestiona las entradas y la salida del sistema. Gestiona la lógica de la aplicación y actúa como intermediario entre el Modelo y la Vista.
2. **Modelo (Model)**: uno o varios modelos que se encargan de buscar los datos e información necesaria. Maneja la lógica de datos y la interacción con la base de datos.
3. **Vista (View)**: una interfaz que muestra los resultados al usuario final. Presenta la interfaz de usuario (UI) y muestra los datos al usuario.



Es muy usado en el desarrollo web porque al tener que interactuar varios lenguajes para crear un sitio es muy fácil generar confusión entre cada componente si estos no son separados de la forma adecuada. Este patrón permite modificar cada uno de sus componentes si necesidad de afectar a los demás.

Laravel implementa cada uno de estos componentes de una manera clara, ayudando a mantener una separación de responsabilidades.

5. MODELO

Eloquent es el ORM (Object Relational Mapper) de Laravel. Permite trabajar con bases de datos usando clases y objetos en lugar de escribir SQL a mano.

Cada tabla se representa con un modelo.

Cada fila de la tabla es una instancia del modelo.

Algunos conceptos importantes en Eloquent:

1. Migraciones

- No son necesarias si tienes las tablas ya creadas en la base de datos.
- Son una forma de definir la estructura de las tablas y el historial de cambios en la base de datos desde la aplicación.
- Se gestionan con comandos (*php artisan migrate*) y se vinculan con los modelos.
- Las migraciones funcionan como un historial del diseño de la base de datos.
- Si necesitas migrar tu aplicación a otro entorno (e.g., local, producción), puedes recrear el esquema fácilmente con *php artisan migrate*.
- Agregar columnas, relaciones, índices o realizar modificaciones es más sencillo y está documentado en el código.

2. Modelos

- Un modelo representa una tabla de la base de datos y permite trabajar con los registros usando Eloquent.
- Cada instancia de un modelo corresponde a una fila de la tabla.
- Se crean extendiendo la clase `Illuminate\Database\Eloquent\Model`.

3. Relaciones

En Laravel, las relaciones permiten que los modelos se **conecten entre sí** de forma sencilla. Esto es súper útil para trabajar con bases de datos que tienen tablas relacionadas (por ejemplo, productos y categorías).

En lugar de hacer consultas SQL complicadas, Laravel te deja definir estas relaciones en los modelos para poder acceder fácilmente a los datos relacionados.

4. Consultas

- Utiliza una sintaxis fluida y expresiva.
- Métodos comunes:
 - Obtener todos los registros → `$users = User::all();`
 - Filtrar registros →
`$user = User::where('email', 'example@mail.com')->first();`
 - Crear nuevos registros →

```
$user = new User;  
$user->name = 'John Doe';  
$user->save();
```

Migraciones

1. Define las migraciones

Laravel crea automáticamente ficheros de migraciones, que si los ejecutamos:

```
php artisan migrate
```

```
INFO Preparing database.
Creating migration table ..... 19ms DONE
INFO Running migrations.
2014_10_12_000000_create_users_table ..... 37ms DONE
2014_10_12_100000_create_password_resets_table ..... 54ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 31ms DONE
2019_12_14_000001_create_personal_access_tokens_table ..... 53ms DONE
```

Crear  varias tablas en la BBDD: users, password_reset_tokens, sessions, cache,...

En la tabla migrations guarda un registro por cada fichero de migraci n ejecutado.

Para crear las tablas de nuestra aplicaci n en la base de datos:

Es posible crear  nicamente la migraci n (m s adelante se ve la creaci n del modelo y a la vez la migraci n) →

php artisan make:migration nombreTabla

Esto crea una migraci n para la tabla sin necesidad de un modelo asociado.

Edita el archivo de migraci n generado en database/migrations/ para definir la estructura de la tabla, as  como sus claves ajenas, cuando las haya.

El modelo se crea por separado cuando lo necesites. Si no necesitas interactuar con la tabla a trav s de Eloquent, no es obligatorio crear el modelo.

IMPORTANTE: Se crea un migraci n por cada tabla f sica que exista en la base de datos, incluyendo tablas normales y tablas intermedias (pivot) de relaciones N:M.

Cada migraci n tiene dos m todos principales: up() y down():

- El m todo up() es donde defines los cambios que quieres hacer en la base de datos, como la creaci n de tablas, la adici n de columnas, o la creaci n de  ndices.

Cuando ejecutas el comando **php artisan migrate**, Laravel llama al m todo up() para aplicar los cambios definidos dentro de  l.

- El m todo down() es donde defines c mo revertir los cambios hechos en el m todo up(). Es decir, si alguna vez necesitas deshacer una migraci n, down() contiene las instrucciones de c mo hacerlo (generalmente eliminando las tablas o columnas creadas).

Si ejecutas **php artisan migrate:rollback**, Laravel llama al método `down()` de la última migración para deshacer los cambios.

Para revertir todas las migraciones ejecuta → **php artisan migrate:reset**

Para eliminar todas las tablas y volver a aplicar todas las migraciones ejecuta →

php artisan migrate:fresh

Ejemplo de migración para la tabla users:

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
});
```

Donde los campos:

`email_verified_at` indica la fecha y hora en la que el usuario ha verificado (por email) su email.

`remember_token` es utilizado por Laravel para recordar la sesión de un usuario entre sesiones.

Convenciones en las migraciones (y modelos):

1. Laravel asume que el nombre de la tabla será el plural y en snake_case (minúsculas y _ entre palabras)
2. La clave primaria de la tabla será id, de tipo bigInteger sin signo (unsignedBigInteger), autoincremental.
3. Las tablas tendrán las columnas `created_at` y `updated_at` para manejar las fechas de creación y actualización de los registros.

En el caso de que la tabla no siga estas convenciones, se indicará en el modelo (explicado más tarde)

4. La clave foránea será el nombre de la tabla a la que se refiere en minúsculas seguido de `_id`.

Si la clave foránea tiene un nombre diferente, puedes especificarlo explícitamente en la migración:

```
$table->foreign('campoFK')
    ->references('campoPKEnOtraTabla')
    ->on('otraTabla')
    ->onDelete('cascade');
```

Supongamos una tabla `profiles` relacionada con `users` con una relación

```
public function up()
{
    Schema::create('profiles', function (Blueprint $table) {
        $table->id();
        $table->unsignedBigInteger('user_id'); // Clave foránea a 'users'
        $table->string('avatar')->nullable();
        $table->timestamps();

        // Definir la clave foránea
        $table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');
    });
}
```

Algunas características que se pueden agregar a las columnas:

- increments() o bigIncrements(): indica que una columna será la PK y autoincrementable.
- unique(): Hace que el valor de la columna sea único en la tabla.

\$table->string('email')->unique();

- nullable(): Permite que la columna acepte valores NULL.
- default(): Define un valor por defecto para la columna.

\$table->integer('status')->default(1);

- primary(): Marca una columna como clave primaria.
- enum(): Define un campo que puede tener un conjunto limitado de valores.

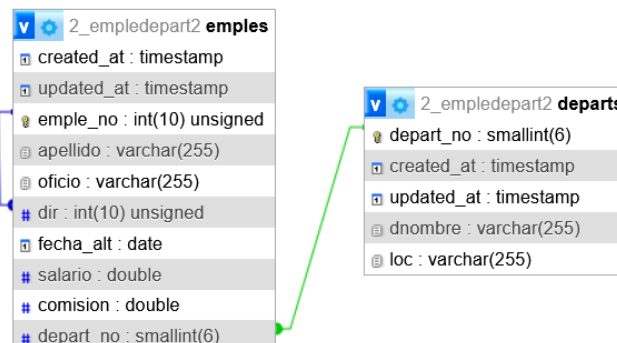
\$table->enum('role', ['admin', 'user', 'guest']);

- foreignId(): Agrega una columna de clave foránea usando un método más conciso.

\$table->foreignId('department_id')->constrained()->onDelete('cascade');

El método constrained() asume que la columna se llamará table_name_id y que hace referencia a la columna id de la tabla correspondiente.

Ejercicio: Realiza un nuevo proyecto donde crearás las tablas EMPLE y DEPART mediante migraciones.



Seeders (sembradores)

Un Seeder sirve para rellenar una tabla con datos iniciales, que pueden ser:

- Datos ficticios (usando factories)
- Datos reales (por ejemplo, valores iniciales que siempre deben existir)
- Datos de configuración (categorías, roles, permisos...)

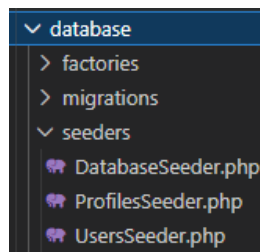
Creamos los ficheros Seeders (insertar datos en las tablas)

Permiten crear registros de prueba de forma automática en las tablas de la base de datos.

○ Creación de un Seeder:

php artisan make:seeder UsersSeeder

Se crea el fichero indicado:



Abre el archivo generado database/seeders/UserSeeder.php y editálo para incluir las sentencias de inserción de datos. Utilizamos DB::statement o bien DB::insert para insertar por separado o juntos los registros de prueba:

```
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Str;

class UsersSeeder extends Seeder
{
    public function run(): void
    {
        $rememberToken1 = Str::random(10);
        $rememberToken2 = Str::random(10);
        $rememberToken3 = Str::random(10);

        $password = bcrypt('123');

        // Insertar los tres registros por separado
        DB::insert("INSERT INTO users (name, email, email_verified_at, password, remember_token, created_at, updated_at)
        VALUES ('Juan Pérez', 'juan@mail.es', now(), '$password', '$rememberToken1', now(), now())");

        DB::insert("INSERT INTO users (name, email, email_verified_at, password, remember_token, created_at, updated_at)
        VALUES ('Elisa López', 'elisa@mail.es', now(), '$password', '$rememberToken2', now(), now())");

        DB::insert("INSERT INTO users (name, email, email_verified_at, password, remember_token, created_at, updated_at)
        VALUES ('María González', 'maria@mail.es', now(), '$password', '$rememberToken3', now(), now())");
    }
}
```

Ejecutamos fuera de la consulta SQL insert los métodos Str::random(10) así como bcrypt('123').

O bien juntos los registros:

```
// Insertar los tres registros en la misma instrucción
DB::statement("INSERT INTO users (name, email, email_verified_at, password, remember_token, created_at, updated_at) VALUES
('Juan Pérez', 'juan@mail.es', now(), '$password', '$rememberToken1', now(), now()),
('Elisa López', 'elisa@mail.es', now(), '$password', '$rememberToken2', now(), now()),
('María González', 'maria@mail.es', now(), '$password', '$rememberToken3', now(), now())");
```

Seeder para profiles:

```
php artisan make:seeder ProfilesSeeder
```

```
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class ProfilesSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        DB::statement('
            INSERT INTO profiles (user_id, avatar, created_at, updated_at) VALUES
            (1, "avatar1.jpg", NOW(), NOW()),
            (2, "avatar2.jpg", NOW(), NOW()),
            (3, "avatar3.jpg", NOW(), NOW())');
    }
}
```

○ Registrar los Seeder:

En el archivo database/seeds/DatabaseSeeder.php, agrega una referencia a los seeders que creaste:

```
<?php

namespace Database\Seeders;

// use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     *
     * @return void
     */
    public function run()
    {
        // \App\Models\User::factory(10)->create();

        // \App\Models\User::factory()->create([
        //     'name' => 'Test User',
        //     'email' => 'test@example.com',
        // ]);
        $this->call(ProductsTableSeeder::class);
        $this->call(CategoriesTableSeeder::class);
    }
}
```

○ Ejecutar los Seeder:

```
php artisan db:seed
```

O bien lanzar un seeder concreto →

```
php artisan db:seed --class=ProfilesSeeder
```

Si quieres borrar toda la información de la base de datos para comenzar desde 0 tendrías que ejecutar:

- php artisan migrate:fresh
- php artisan db:seed

Ejercicio: Realiza seeders para emple y depart con los registros indicados en el aula virtual.

Modelos

Se encuentran en el directorio app/Models

Los modelos en Eloquent representan tablas en tu base de datos.

Si tenemos tablas intermedias de relaciones N:M que no aportan información adicional (solo tienen las FK de las tablas relacionadas), no es necesario crearles un modelo, ya que Eloquent permite acceder a los datos relacionados de ambas tablas sin necesidad de crear modelo para la tabla intermedia.

Crea los Modelos (con o sin sus migraciones)

Cada modelo hereda de Illuminate\Database\Eloquent\Model.

```
php artisan make:model Depart
```

Con la opción **-m** crea automáticamente el archivo de migración, para crear la tabla en la Base de Datos.

Por convención, el modelo representa una tabla en plural (p.ej. Depart representa a la tabla departs)

Esto genera:

1. El modelo Depart en app/Models/Depart.php.
2. Una migración en database/migrations/xxxx_xx_xx_create_departs_table.php.

Modelo Dept → app/Models/Depart.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Depart extends Model
{
    use HasFactory;

    protected $table = 'depart'; // Nombre de la tabla en la base de datos
    protected $primaryKey = 'depart_no'; // Clave primaria

    public $incrementing = false; // Si la clave primaria no es autoincremental
    protected $fillable = ['depart_no', 'dnombre', 'loc']; // Columnas que se pueden rellenar
}
```

La vble protected \$fillable se debe definir para indicarle los campos de la tabla masivamente asignables, aparte de id y los timestamps (fecha y hora de creación y fecha y hora de última modificación).

El resto de datos del ejemplo se definen porque no se siguen las convenciones:

- El nombre del modelo va en singular y PascalCase (la primera letra de cada palabra esté en mayúscula sin excepciones, estando el resto de letras en minúsculas).
- El nombre de la tabla va en plural y snake_case.

- La PK es una columna llamada id de tipo BIGINT, auto-incremental y unsigned.
- Campos created_at y updated_at se gestionan automáticamente si existen en la tabla. Se pueden desactivar con → **public \$timestamps = false;**
- El nombre del archivo del modelo debe coincidir con el nombre de la clase, y ubicarse en app/Models/NombredelModelo.php

El uso de HasFactory está relacionado con el uso de factories para generar datos de prueba de forma fácil y eficiente. Esta característica es especialmente útil cuando estás trabajando con pruebas o seeding (llenar la base de datos con datos iniciales).

Configura las siguientes propiedades según tu tabla:

- **Tabla personalizada (si el nombre no sigue la convención plural):**

```
protected $table = 'categories';
```

- **Clave primaria personalizada (si no es id):**

```
protected $primaryKey = 'post_id';
```

- **Desactivar incremento automático (si usas claves no incrementales):**

```
public $incrementing = false;
```

- **Tipo de la clave primaria (si no es un entero):**

```
protected $keyType = 'string';
```

- **Campos asignables en masa (\$fillable):** Define qué atributos pueden asignarse en masa. La asignación masiva ocurre cuando se pasa un array asociativo (al modelo, en lugar de asignar cada atributo uno por uno).

```
protected $fillable = ['name', 'description'];
```

Los campos created_at y updated_at son gestionados automáticamente por Laravel cuando usas el **trait** Timestamps (habilitado por defecto en los modelos). No se asignan directamente en la creación o actualización de registros, ya que Laravel los genera automáticamente al insertar o actualizar registros en la base de datos.

- **Campos protegidos (\$guarded):** Alternativa a \$fillable, especifica qué atributos no deben asignarse en masa:

```
protected $guarded = ['id'];
```

En lugar de especificar qué campos se pueden asignar, con \$guarded defines qué campos **no** pueden ser asignados. Todo lo demás será asignable.

Relaciones en el modelo

Si la tabla tiene relaciones con otras tablas, se pueden definir como métodos en el modelo:

- **Uno a uno (hasOne, belongsTo)**

Una fila en una tabla está relacionada con exactamente una fila en otra tabla.

Por ejemplo: Un usuario tiene un perfil:

User (user_id, username,...) (1) (hasOne) ----- (1)(belongsTo)Profile (profile_id, name, user_id(FK))

Modelo User:

```
public function profile()
{
    return $this->hasOne(Profile::class);
}
```

Modelo Profile:

```
public function user()
{
    return $this->belongsTo(User::class);
}
```

- **Uno a muchos (hasMany, belongsTo)**

Una fila en una tabla está relacionada con una o varias filas en otra tabla.

Ejemplo: un producto es una de una única categoría, pero cada categoría puede tener varios productos asignados.

```
class Category extends Model
{
    use HasFactory;

    protected $fillable = ['name', 'description'];

    public function product()
    {
        return $this->hasMany(Product::class);
    }
}
```

```
class Product extends Model
{
    protected $fillable = ['name', 'description', 'price', 'forsale', 'type'];

    protected $casts = [
        'price' => 'float',
        'forsale' => 'boolean',
        'category_id' => 'integer',
    ];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }
}
```

Una vez definida la relación, si queremos recuperar todos los productos, cargando también la categoría asociada a cada uno de ellos en una única consulta (Eager Loading o carga relacionada), se puede hacer con:

Product::with('category')->get();

Sin esto, tendrías el problema de que por cada producto haría otra consulta para traer su categoría. Con `with('category')`, lo soluciona.

Y accederíamos a los datos:

```
$products = Product::with('category')->get();

foreach ($products as $product) {
    echo $product->name;
    echo $product->category->name; // Aquí accedes a la categoría relacionada
}
```

Otro ejemplo:

```
public function posts() {
    return $this->hasMany(Post::class);
}
```

```
public function profile() {
    return $this->hasOne(Profile::class);
}
```

Estos ejemplos se encontrarían dentro de un modelo Users, indicando que cada usuario puede tener varios posts (publicaciones) y un único perfil.

No se indican los nombres de los campos FK de estas tablas ya que Laravel asume que el campo de la clave foránea en los modelos será el nombre de la tabla en minúsculas con el sufijo `_id`. En caso de no seguir esta convención, habría que poner el campo de la tabla FK en esa relación.

○ Muchos a muchos (`belongsToMany`)

Una fila en una tabla está relacionada con muchas filas en otra tabla, y viceversa. Esto generalmente se representa con una tabla intermedia. Esta tabla intermedia se ha de crear en la Base de Datos (migración), pero no necesariamente en el modelo, ya que Laravel permite trabajar directamente con las dos tablas relacionadas (`belongsToMany`).

Si la tabla intermedia tiene atributos adicionales, es necesario realizar un modelo para dicha tabla.

Ejemplo: un producto puede tener varios alérgenos y viceversa.

Por defecto, Laravel espera que la tabla intermedia se llame `allergen_product`, en singular, en orden alfabético y unidos por un guión bajo. Puedes personalizarlo pasando el nombre de la tabla intermedia como parámetro.

En Laravel, no necesitas crear un modelo para la tabla intermedia en una relación muchos a muchos (N:M), a menos que quieras acceder directamente a esa tabla o guardes datos adicionales en ella (como una columna `created_at`, cantidad, etc.).

```
class Allergen extends Model
{
    protected $table = 'allergens'; // Nombre de la tabla en la base de datos
    protected $fillable = ['name', 'description']; // Campos que se pueden llenar masivamente

    public function products()
    {
        //return $this->belongsToMany(Product::class);

        //pero al llamarse allergens_products
        //es necesario especificar el nombre de la tabla intermedia
        $this->belongsToMany(Product::class, 'allergens_products');

        //Si además, tuviera nombres de columnas personalizados
        //return $this->belongsToMany(Allergen::class, 'allergens_products', 'id_alergeno', 'id_allergen');
    }
}
```

```
class Product extends Model{
    protected $fillable = [
        'name', 'description', 'price', 'image', 'available_for_takeaway', 'type', 'category_id'
    ];

    public function getImagenBase64(){
        return base64_encode($this->image);
    }

    public function allergens(){
        //si se llamase la tabla intermedia allergen_product
        //return $this->belongsToMany(Allergen::class);

        //pero al llamarse allergens_products
        //rompe la convención por defecto de Laravel, y hay que especificarlo manualmente en el modelo
        return $this->belongsToMany(Allergen::class, 'allergens_products');
    }
}
```

Realizar operaciones con el Modelo

Puedes usar las funciones de Eloquent para interactuar con la base de datos.

Crear un nuevo registro:

Lo realizamos de forma explícita, donde puede quedar más claro:

```
public function createDepart($a, $b, $c){
    try{
        $depart = new Depart();
        $depart->depart_no = $a;
        $depart->dname = $b;
        $depart->loc = $a;
        $depart->save(); // Guardar en la base de datos
        error_log('Departamento creado');
    }catch(\Exception $e){
        error_log('Error al crear el departamento: '.$e);
    }
}
```

O bien de forma implícita con create:

```
try {
    Depart::create([
        'depart_no' => $a,
        'dnombre'   => $b,
        'loc'        => $c,
    ]);
} catch (\Exception $e) {
    die("Error al insertar departamento: " . $e->getMessage());
}
```

Obtener todos los registros:

```
public function getDeparts(){
    try{
        $departs = Depart::all();
        return $departs;
    }catch(\Exception $e){
        error_log('Error al obtener los departamentos: '.$e);
    }
}
```

Obtener un determinado registro por id:

```
public function getDepart($id){
    try{
        $depart = Depart::find($id);
        return $depart;
    }catch(\Exception $e){
        error_log('Error al obtener el departamento: '.$e);
    }
}
```

Obtener un registro según una condición:

```
Depart::where('loc', '=', 'Madrid')->get();
```

Actualizar un registro:

```
public function updateDepart($id, $a, $b, $c){
    try{
        $depart = $this->getDepart($id);
        $depart->depart_no = $a;
        $depart->dnome = $b;
        $depart->loc = $c;
        $depart->save();
        error_log('Departamento actualizado');
    }catch(\Exception $e){
        error_log('Error al actualizar el departamento: '.$e);
    }
}
```

O bien con update:

```
Depart::find($id)->update([
    'depart_no' => $a,
    'dname'    => $b,
    'loc'      => $c,
]);
```

Eliminar un registro:

```
public function deleteDepart($id){
    try{
        $depart = $this->getDepart($id);
        $depart->delete();
        error_log('Departamento eliminado');
    }catch(\Exception $e){
        error_log('Error al eliminar el departamento: '.$e);
    }
}
```

Depart::where('loc', '=', 'Madrid')->delete(); // por condición

O bien con destroy:

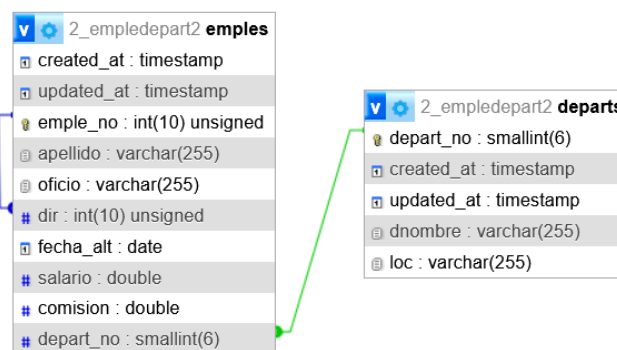
Depart::destroy(1); // por ID

Una relación más amplia de operaciones a realizar con el modelo:

Operación	Código	Descripción
Obtener todos los registros	<code>Product::all();</code>	Obtiene todos los registros de la tabla products.
Obtener por ID	<code>Product::find(\$id);</code>	Obtiene un registro por su ID o null si no existe.
Obtener por ID o fallar	<code>Product::findOrFail(\$id);</code>	Obtiene un registro por su ID o lanza una excepción si no existe.
Filtrar con where	<code>Product::where('price', '>', 100)->get();</code>	Filtra productos cuyo precio sea mayor a 100.
Obtener el primero	<code>Product::where('forsale', true)->first();</code>	Obtiene el primer producto que está a la venta.
Ordenar resultados	<code>Product::orderBy('price', 'asc')->get();</code>	Ordena los productos por precio en orden ascendente.
Limitar resultados	<code>Product::take(5)->get();</code>	Obtiene los primeros 5 productos.
Paginar resultados	<code>Product::paginate(10);</code>	Trae los resultados paginados, en bloques de 10 productos por página.
Contar registros	<code>Product::count();</code>	Cuenta el número total de productos.

Operación	Código	Descripción
Crear un registro	<code>Product::create(['name' => 'Laptop', 'price' => 1000]);</code>	Crea un nuevo producto en la tabla products.
Actualizar un registro	<code>\$product = Product::find(\$id); \$product->update(['price' => 1200]);</code>	Actualiza el precio de un producto existente.
Eliminar por instancia	<code>\$product = Product::find(\$id); \$product->delete();</code>	Elimina un producto existente.
Eliminar directamente	<code>Product::destroy(\$id);</code>	Elimina un producto por ID.
Eliminar por condición	<code>Product::where('price', '<', 10)->delete();</code>	Elimina todos los productos cuyo precio sea menor a 10.
Cargar relaciones	<code>Product::with('category')->get();</code>	Carga la relación category definida en el modelo Product.
Obtener un producto con su categoría	<code>\$product = Product::with('category')->findOrFail(\$id);</code>	
Validar existencia	<code>Product::where('name', 'Laptop')->exists();</code>	Comprueba si existe al menos un producto con el nombre Laptop.
Actualizar en lote	<code>Product::where('forsale', false)->update(['enVenta' => true]);</code>	Marca como "enVenta" todos los productos que no están a la venta.
Eliminar en lote	<code>Product::where('enVenta', false)->delete();</code>	Elimina todos los productos que no están a la venta.
Crear relacionado	<code>\$product->category()->associate(\$category)->save();</code>	Asocia un producto a una categoría existente y lo guarda.
Consultar con Scope	<code>Product::expensive()->get();</code>	Usa un scope definido en el modelo para filtrar productos caros.
Manejo de excepciones	<pre>try { \$product = Product::findOrFail(\$id); ... } catch (ModelNotFoundException \$e) { ... }</pre>	Maneja errores si un producto no es encontrado.
Consultas crudas	<code>Product::whereRaw('LENGTH(name) > ?', [10])->get();</code>	Ejecuta una consulta SQL personalizada.

Ejercicio: Realiza los modelos de las tablas EMPLE y DEPART.



6. CONTROLADOR

Los controladores te permiten organizar el código de manera limpia y separar la lógica del manejo de rutas y vistas.

Crear Controlador

El controlador generado estará ubicado en `app/Http/Controllers`

- Crear un controlador básico:

```
php artisan make:controller DepartController
```

app/Http/Controllers/DepartController.php:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class DepartController extends Controller
{
    //
}
```

Editamos este archivo:

- Importamos el modelo Depart:

```
use App\Models\Depart;
```

- Creamos el método que será llamado desde el router (`/routes/web.php`)

```
public function index(){
```

- Creamos un objeto del modelo:

```
$depart = new Depart(); // Crear un objeto del modelo
```

- Llamamos al método del modelo que recupera los registros:

```
$departs = $depart->getDeparts();
```

- Pasamos estos datos a una vista:

```
return view('depart.index', compact('departs'));
```

El método `view()` es una función que se utiliza para renderizar una vista Blade. Recibe un parámetro en forma de cadena que indica la ubicación de la vista (carpeta.archivo), lo que se traduce en la ruta `resources/views/carpeta/archivo.blade.php`

- 'depart.index' se traduce en resources/views/depart/index.blade.php.
- 'home' se traduce en resources/views/home.blade.php.

Si la vista no existe, Laravel lanzará un error.

El método `compact()` es una función de PHP que crea un array asociativo a partir de una o más variables. En este caso crea un array asociativo con el nombre de la variable `$departs` como clave y su valor como valor.

Por ejemplo, si la variable `$departs` tiene este valor:

```
$departs = [  
    ['depart_no' => 1, 'dnombre' => 'Marketing', 'loc' => 'Madrid'],  
    ['depart_no' => 2, 'dnombre' => 'Ventas', 'loc' => 'Barcelona']  
];
```

Entonces `compact('departs')` generará:

```
[  
    'departs' => [  
        ['depart_no' => 1, 'dnombre' => 'Marketing', 'loc' => 'Madrid'],  
        ['depart_no' => 2, 'dnombre' => 'Ventas', 'loc' => 'Barcelona']  
    ]  
]
```

Este array se pasa a la vista como "datos" que puede usar.

El código:

```
$categories = Category::all();  
return view('products.create', compact('categories'));
```

Es equivalente a:

```
$categories = Category::all();  
return view('products.create', ['categories' => $categories]);
```

La palabra clave `return` envía la respuesta generada al navegador del cliente. En este caso, la respuesta es el contenido HTML generado por la vista `depart.index`, después de haber procesado los datos (`departs`).

Laravel procesa los datos de la siguiente forma:

1. Carga la vista `depart.index`.
2. Pasa el array `compact('departs')` como datos a la vista.
3. Renderiza la vista sustituyendo las variables en el archivo Blade (como `@foreach` o `{{ }}`) con los datos proporcionados.
4. Devuelve el HTML generado al cliente.

7. CONFIGURAR LAS RUTAS

Para que el controlador funcione, debes agregarlo a las rutas. Esto se hace en los archivos routes/web.php o routes/api.php, dependiendo de si es una ruta web o API.

Definir rutas básicas:

```
use App\Http\Controllers\NombreDelControlador;
```

```
Route::get('/ruta', [NombreDelControlador::class, 'metodo']);
```

En el ejemplo que estamos realizando, es en el fichero web.php donde hay que definir la ruta para acceder al controlador desde el navegador:

routes/web.php

```
use App\Http\Controllers\DepartController;
```

```
Route::get('/depart', [DepartController::class, 'index']);
```

Esto vincula la ruta /depart con el método index del controlador DepartController.

Poner nombre a las rutas

Si queremos ponerle un nombre a una ruta para luego direccionar a esta ruta:

/route/web.php:

```
Route::get('/reservas', [ReservaController::class, 'index'])->name('reservas.index');
```

```
Route::post('/reservas', [ReservaController::class, 'create'])->name('reservas.create');
```

/app/Http/Controller/xxxController.php:

Redirección a otra ruta sin mandarle datos:

```
return redirect()->route('reservas.index');
```

Redirección mandándoles datos:

```
return redirect()->route('reservas.index', ['fecha' => $request->fecha]);
```

Desde las vistas:

Action de un formulario:

```
<form method="POST" action="{{ route('reservas.create') }}">
```

Action de un formulario enviando datos:

```
<form method="POST" action="{{ route('reservas.destroy', $reserva->id) }}">
```

Ejercicio: Crea un CRUD para los modelos EMPLE y DEPART.

Departamentos

[index](#) →

[Crear departamento](#)

- CONTABILIDAD - SEVILLA [Editar](#) [Eliminar](#)
- INVESTIGACIÓN - MADRID [Editar](#) [Eliminar](#)
- VENTAS - BARCELONA [Editar](#) [Eliminar](#)
- PRODUCCIÓN - BILBAO [Editar](#) [Eliminar](#)

Crear Departamento

[create](#) →

Número de departamen	Nombre del departamento	Ubicación	Guardar
---	--	--	--

[Volver](#)

Una vez guardado el nuevo registro, vuelve al index

Editar Departamento

[edit](#) →

10	CONTABILIDAD	SEVILLA	Actualizar
---------------------------------	---	--------------------------------------	---

[Volver](#)

Una vez actualizado el registro, vuelve al index

[delete](#) → Borra el registro y redirecciona al index

En el formulario de nuevo EMPLEAD@, en la opción de dir (director/a), pon un select donde se muestre el emple_no – apellido del resto de empleados que puedan ser su director/a.

Del mismo modo, en este formulario, en lugar de poner únicamente el depart_no, muestra también en un select el dnombre de los departamentos.

8. VISTAS DINÁMICAS

En Laravel, las vistas son archivos que contienen la presentación de la aplicación, típicamente en formato HTML con la posibilidad de incluir código PHP embebido. Estas vistas están diseñadas para trabajar con datos que se les pasan desde los controladores. Las vistas están ubicadas, por defecto, en el directorio **resources/views**.

Laravel usa Blade como su motor de plantillas.

Crea un archivo de vista para mostrar los datos en el navegador. Crea el archivo `resources/views/depart/index.blade.php` con el siguiente contenido:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Departamentos</title>
    <link rel="stylesheet" href="{{ asset('css/styles.css') }}">
</head>
<body>
    <h1>Lista de Departamentos</h1>
    <table>
        <thead>
            <tr>
                <th>ID</th>
                <th>Nombre</th>
                <th>Localización</th>
            </tr>
        </thead>
        <tbody>
            @foreach($departs as $depart)
                <tr>
                    <td>{{ $depart->DEPART_no }}</td>
                    <td>{{ $depart->dnombre }}</td>
                    <td>{{ $depart->loc }}</td>
                </tr>
            @endforeach
        </tbody>
    </table>
</body>
</html>
```

La línea

```
<link rel="stylesheet" href="{{ asset('css/styles.css') }}">
```

es utilizada en Laravel para **cargar un archivo CSS desde el directorio público del proyecto**. En Laravel, la función **asset()** genera una URL completa hacia un archivo ubicado en el directorio `public/` del proyecto.

- **Crear una vista:** Una vista es un archivo `.blade.php`.

/resources/views/bienvenida.blade.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi aplicación Laravel</title>
</head>
<body>
  <h1>Bienvenido a mi aplicación Laravel</h1>
</body>
</html>
```

- Laravel utiliza la notación de puntos (.) para especificar la ubicación de las vistas. El punto (.) reemplaza el carácter / o \.

- **Renderizar una vista desde un controlador:** Puedes devolver una vista desde un controlador usando el método view.

App/Http/Controller.php

```
public function index(){
    return view('bienvenida');
}
```

O bien, si le pasamos un array asociativo con datos:

```
public function index(){
    return view('bienvenida', ['nombre'=> 'Laravel']);
}
```

Mientras que en resources/views/bienvenida.blade.php:

```
<body>
  <h1>Bienvenido a mi aplicación {{ $nombre }}</h1>
</body>
```

localhost:8000/bienvenida

Bienvenido a mi aplicación Laravel

- **Actualizar las rutas en routes/web.php.** Agrega una nueva ruta que apunte al método index de tu controlador:

```
use App\Http\Controllers\Controller;
```

```
Route::get('/bienvenida', [Controller::class, 'index']);
```

Blade incluye una amplia variedad de directivas para trabajar con datos y lógica de manera eficiente en las vistas:

@foreach

Se utiliza para iterar sobre un conjunto de datos, como una colección o un array.

```
@foreach ($items as $item)
    <p>{{ $item }}</p>
@endforeach
```

Ejemplo básico:

En el controlador:

```
return view('ejemplo', ['nombres' => ['Juan', 'Ana', 'Luis']]);
```

En la vista Blade:

```
<ul>
    @foreach ($nombres as $nombre)
        <li>{{ $nombre }}</li>
    @endforeach
</ul>
```

Salida HTML:

```
<ul>
    <li>Juan</li>
    <li>Ana</li>
    <li>Luis</li>
</ul>
```

@if, @elseif, @else

Se utilizan para manejar condiciones en Blade.

```
@if ($condition)
    <p>Condición verdadera</p>
@elseif ($otherCondition)
    <p>Otra condición verdadera</p>
@else
    <p>Ninguna condición es verdadera</p>
@endif
```

Ejemplo básico:

```
@if (count($nombres) > 0)
    <p>Tenemos nombres:</p>
    <ul>
        @foreach ($nombres as $nombre)
            <li>{{ $nombre }}</li>
        @endforeach
    </ul>
@else
    <p>No hay nombres disponibles.</p>
@endif
```

En la vista Blade

@isset y @empty

Verifican si una variable está definida o vacía.

@isset → Se usa para comprobar si una variable está definida y no es null.

```
@isset($nombre)
    <p>El nombre es {{ $nombre }}</p>
@endisset
```

@empty → Se usa para comprobar si una variable está vacía (null, "", array vacío, etc.).

```
@empty($items)
    <p>No hay elementos disponibles.</p>
@endempty
```

@for y @while

```
@for ($i = 0; $i < 5; $i++)
    <p>Iteración {{ $i }}</p>
@endfor
```

```
@php $i = 0; @endphp
@while ($i < 3)
    <p>Iteración {{ $i }}</p>
    @php $i++; @endphp
@endwhile
```

@switch y @case

```
@switch($tipo)
    @case('admin')
        <p>Bienvenido, administrador</p>
        @break

    @case('usuario')
        <p>Bienvenido, usuario</p>
        @break

    @default
        <p>Tipo de usuario desconocido</p>
@endswitch
```

@include

Se utiliza para incluir otras vistas dentro de la vista actual.

```
@include('header') <!-- Incluye la vista header.blade.php -->
@include('footer', ['year' => 2025]) <!-- Pasa datos a la vista footer -->
```

@yield y @section

Se utilizan para definir y mostrar secciones de contenido en layouts (plantillas).

En el layout:

```
<html>
<head>
    <title>@yield('titulo')</title>
</head>
<body>
    @yield('contenido')
</body>
</html>
```

@csrf

En la vista:

```
@extends('layout')

@section('titulo', 'Página Principal')

@section('contenido')
    <p>Contenido de la página principal.</p>
@endsection
```

Esta directiva agrega un campo de token CSRF (protección contra ataques de falsificación de solicitudes) en formularios.

CSRF significa Cross-Site Request Forgery, o en español Falsificación de solicitudes entre sitios. Es un tipo de ataque que engaña a un usuario autenticado para que realice acciones no deseadas en una aplicación web en la que ya ha iniciado sesión.

¿Cómo funciona?

1. Supongamos que estás logueado en tu banca en línea (ej. banco.com).
2. Sin cerrar sesión, visitas un sitio malicioso (sitio-malicioso.com).
3. Ese sitio malicioso carga una solicitud (por ejemplo, un formulario oculto que hace una transferencia) apuntando a banco.com.
4. Como ya tienes la sesión iniciada, el navegador incluye automáticamente tus cookies (como la de sesión) en la solicitud.
5. Resultado: Se ejecuta una acción en tu cuenta sin tu consentimiento.

El token CSRF (Cross-Site Request Forgery token) es una medida de seguridad implementada por muchas aplicaciones web (como Laravel) para prevenir estos ataques.

¿Qué hace el token?

1. Cada vez que se carga un formulario, la app incluye un token CSRF oculto en el HTML.
2. Cuando el formulario se envía, el servidor espera ese mismo token en la solicitud.
3. Si el token no coincide o no está presente, el servidor rechaza la solicitud.

¿Por qué funciona?

Porque un sitio malicioso no puede obtener ni enviar correctamente ese token CSRF (está generado para cada sesión/usuario y no es accesible desde otro dominio). Así, aunque el navegador incluya las cookies automáticamente, falla la validación del token.

En la vista:

```
<form action="/ruta" method="POST">
  @csrf
  <input type="text" name="nombre">
  <button type="submit">Enviar</button>
</form>
```

Genera automáticamente:

```
<input type="hidden" name="_token" value="token_csrf_generado">
```

@auth

Muestra contenido solo si el usuario está autenticado con sesiones nativas de Laravel:

```
@auth
    <p>Bienvenido, {{ auth()->user()->name }}</p>
@endauth
```

Es equivalente a:

```
@if(Auth::check())
    ...
@endif
```

En Laravel, hacen lo mismo las siguientes instrucciones, ambos funcionan igual en Blade y controladores.

Método	Descripción
<code>Auth::user()</code>	Usa la facade <code>Auth</code> , estilo más clásico o estático.
<code>auth()->user()</code>	Usa el helper function <code>auth()</code> , estilo más moderno y funcional.

El dato `auth()->user()->name` proviene del sistema de autenticación de Laravel. Cuando un usuario inicia sesión, Laravel almacena la información del usuario autenticado en la sesión, y se puede acceder a ella mediante `auth()->user()`. El atributo `name` es un campo en la tabla `users`, que Laravel utiliza por defecto para manejar la autenticación.

@guest

Muestra contenido solo si el usuario **no** está autenticado:

```
@guest
    <p>Por favor, inicia sesión.</p>
@endguest
```

@php

Permite ejecutar código PHP directamente en la vista.

```
@php
    $contador = 1;
@endphp

<p>El contador es: {{ $contador }}</p>
```

Combinación de varias directivas

Puedes usar varias directivas juntas para lograr vistas dinámicas. Por ejemplo:

```
<ul>
  @foreach ($productos as $producto)
    @if ($producto->stock > 0)
      <li>{{ $producto->nombre }} (Disponible)</li>
    @else
      <li>{{ $producto->nombre }} (Agotado)</li>
    @endif
  @endforeach
</ul>
```

Layouts en Blade

Los layouts permiten definir una plantilla base que otras vistas pueden extender. Se usa la directiva `@extends` para heredar un layout y `@section` para definir contenido en ciertas secciones.

- `@extends('vistaPlantilla')`, `@yield('seccion')`, `@section('seccion')`

Crear un layout base →

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>@yield('title')</title>
  <link rel="stylesheet" href="{{ asset('css/app.css') }}">
</head>
<body>
  <header>
    <h1>Mi Aplicación</h1>
  </header>

  <main>
    @yield('content')
  </main>

  <footer>
    <p>© 2025 - Todos los derechos reservados</p>
  </footer>
</body>
</html>
```


Aquí, `@yield('title')` y `@yield('content')` son espacios donde otras vistas pueden inyectar contenido.

Extender el layout en una vista

Creemos una vista que extienda el layout, por ejemplo, `resources/views/home.blade.php`

```
@extends('layouts.app')

@section('title', 'Inicio')

@section('content')
    <h2>Bienvenido a mi aplicación</h2>
    <p>Esta es la página principal.</p>
@endsection
```

- **@include: Incluir Vistas**

Para reutilizar fragmentos, usa `@include`.

`/views/partials/navbar.blade.php`

```
<nav>
    <ul>
        <li><a href="/">Inicio</a></li>
        <li><a href="/about">Acerca</a></li>
        <li><a href="/contact">Contacto</a></li>
    </ul>
</nav>
```

En el layout o en cualquier vista, inclúyelo con:

```
@include('partials.navbar')
```

- **@push y @stack para scripts o estilos adicionales**

En la cabecera del layout (`app.blade.php`), agrega:

```
@stack('scripts')
```

Y en una vista hija específica:

```
@push('scripts')
    <script src="{{ asset('js/custom.js') }}"></script>
@endpush
```

- **Mensajes tipo flash:**

Podemos poner en el controlador:

```
return redirect()->route('products.create')->with('success', 'Producto guardado correctamente.');
```

donde:

```
redirect()->route('products.create')
```

Redirige al usuario hacia la ruta con nombre admin.products.create (por ejemplo: /admin/products/create), normalmente el formulario para añadir otro producto.

```
->with('success', 'Producto guardado correctamente.')
```

Añade un mensaje de tipo flash a la sesión con clave 'success' y ese texto como valor. Este mensaje:

- Se guarda solo durante una petición
- Se puede mostrar fácilmente en la vista siguiente (normalmente con session('success'))

Es un tipo de dato temporal que Laravel guarda en la sesión y se elimina automáticamente después de mostrarlo una vez.

Cómo mostrar ese mensaje en tu vista Blade:

```
@if (session('success'))
    <div class="alert alert-success">
        {{ session('success') }}
    </div>
@endif
```

Este bloque comprueba si hay un mensaje success en la sesión, y si lo hay, lo muestra con el estilo de alerta de Bootstrap.

- **Alternativas útiles**

Puedes usar distintos tipos de mensajes:

```
->with('error', 'Algo salió mal.')
```

```
->with('info', 'Este es un aviso informativo.')
```

Y en la vista:

```
@if (session('error'))
```

```
<div class="alert alert-danger">{{ session('error') }}</div>
```

```
@endif
```

Resumen:

Directiva	Función
@extends('layouts.app')	Hereda una plantilla base
@section('nombre')	Define contenido en una sección
@yield('nombre')	Indica dónde se mostrará ese contenido
@include('partials.view')	Incluye una vista parcial
@push('scripts') / @stack('scripts')	Agrega scripts o estilos adicionales
@if, @foreach, @auth, @guest	Control de flujo

Ejercicio: Modifica las vistas del CRUD de EMPLE y DEPART, utilizando partials (header → Emple Depart) y footer (Tu nombre y fecha), dentro de una plantilla (template o layout) en la que se basan las vistas Blade.

9. GESTIÓN DE PARÁMETROS EN LA RUTA Y DATOS EN BODY (POST) CON VALIDACIONES

Usar parámetros en las rutas:

En Laravel, los parámetros en las rutas URL se definen dentro del archivo `routes/web.php`.

Laravel permite definir parámetros obligatorios y opcionales, e incluso aplicar restricciones con expresiones regulares.

1. Parámetros obligatorios. Se definen encerrando el parámetro entre llaves `{}` en la ruta, indicando que la ruta espera un valor.

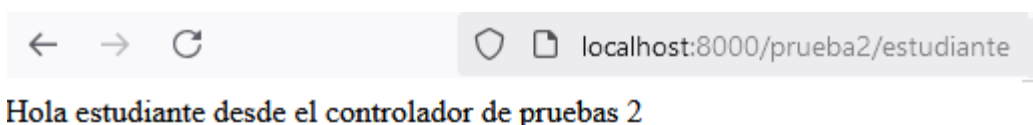
app/Http/Controllers/pruebasController.php:

```
public function prueba2($id){  
    return "Hola ". $id. " desde el controlador de pruebas 2";  
}
```

routes/web.php

```
Route::get('/prueba2/{id}', [pruebasController::class, 'prueba2']);
```

Probamos la ruta:

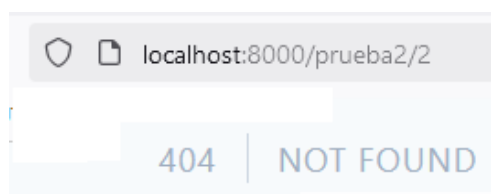


Validación de parámetros

Puedes agregar validaciones a los parámetros directamente en las rutas. En el caso de no cumplir la validación, Laravel devolverá un error 404.

routes/web.php

```
Route::get('/prueba2/{id}', [pruebasController::class, 'prueba2'])->where('id', '[a-z]+');
```



2. Parámetros Opcionales. Si un parámetro puede no estar presente en la URL, se define con un signo `?` y un valor por defecto en la función.

Cuando defines una ruta con parámetros opcionales y ésta llama a un controlador, debes asegurarte de asignar un valor predeterminado al parámetro en la función del controlador.

/routes/web.php

```
Route::get('/prueba2b/{id?}', [pruebasController::class, 'prueba2b']);
```

app/Http/Controllers/pruebasController.php:

```
public function prueba2b($id = "Invitado"){
    return "Hola ". $id. " desde el controlador de pruebas 2b";
}
```

Ejemplo con modelo Eloquent y parámetro opcional

Si quieres que el parámetro sea un Depart_no de departamento y que Laravel haga la búsqueda automática en la base de datos:

/routes/web.php

```
Route::get('/departamento/{dpto?}', [DepartController::class, 'mostrar']);
```

app/Http/Controllers/DepartController.php:

```
public function mostrar(Depart $dpto = null){
    if ($dpto) {
        return "Departamento, " . $dpto->dnombre;
    }
    return "Departamento, NULL";
}
```

Laravel buscará automáticamente el departamento en la base de datos (Depart::find(\$dpto)).

Cuando usamos **Route Model Binding** con un modelo de Eloquent (Depart \$dpto), si el departamento no existe, Laravel devuelve automáticamente un error 404 Not Found en lugar de permitir un valor predeterminado.

Si cuando se pone un departamento no válido o no se pone, no queremos que muestre el 404-Not Found, tenemos que modificar el controlador como sigue:

```
public function mostrar($dpto = null){
    $departamento = $dpto ? Depart::find($dpto) : null;
    if ($departamento) {
        return "Departamento, " . $departamento->dnombre;
    }
    return "No se ha indicado ningún Departamento válido";
}
```

Datos enviados en el cuerpo (POST)

Los controladores pueden acceder a los datos enviados por el cliente, como formularios o datos en el cuerpo de la solicitud.

En Laravel todos los datos enviados desde un formulario (sea input, select, textarea, etc.) se reciben en el objeto **\$request** (Illuminate\Http\Request), y Laravel ofrece varios métodos para acceder a ellos.

- **input**

resources/views/prueba3form.blade.php

```
<form action="/prueba4" method="POST">
    @csrf <!-- Token CSRF necesario -->
    <input type="text" name="nombre" placeholder="Escribe tu nombre">
    <button type="submit">Enviar</button>
</form>
```

routes/web.php

```
Route::get('/prueba3', [pruebasController::class, 'prueba3']);

Route::post('/prueba4', [pruebasController::class, 'prueba4']);
```

app/Http/Controllers/pruebasController.php:

```
public function prueba3(){
    return view('prueba3form');
}

public function prueba4(Request $request){
    $nombre = $request->input('nombre');
    return "Hola ". $nombre. " desde el controlador de pruebas 4";
}
```

Laravel inyecta automáticamente la clase Request, que contiene todos los datos de la solicitud HTTP enviada por el usuario.

`$request->input('nombre')` → Obtiene el valor del campo de formulario con el nombre "nombre".

localhost:8000/prueba3

localhost:8000/prueba4

Hola Profe desde el controlador de pruebas 4

Si quieres validar los datos dados por el usuario, antes de procesar la solicitud, usa validaciones:

```
public function prueba4b(Request $request){
    $request->validate([
        'nombre' => 'required|string|max:5' //si nombre no es string o tiene más de 5 caracteres, se volverá a mostrar el formulario
    ]);
    $nombre = $request->input('nombre');
    return "Hola ". $nombre. " desde el controlador de pruebas 4";
}
```

Si la validación falla, Laravel NO ejecuta el resto del método, hace automáticamente un `redirect back()` y guarda los errores en `$errors`, NO en `session('error')`. Para mostrar los errores deberíamos tener en la vista:

```
{{-- Mostrar errores --}}
@if ($errors->any())
    <ul>
        @foreach ($errors->all() as $error)
            <li style="color:red">{{ $error }}</li>
        @endforeach
    </ul>
@endif
```

- **Acceso directo como propiedad mágica**

`$valor = $request->nombre;`

Funciona igual que `input()`, pero es más corto.

- **get()**

`$valor = $request->get('categorias');`

Es lo mismo que `input()`, solo que más común en PHP en general.

- **all()**

Devuelve todos los datos enviados por el formulario como un array. No incluye archivos subidos, solo campos normales.

`$data = $request->all();` // Devuelve todos los campos como array

`$valor = $data['categorias'];` // Puedes acceder al campo del select

- **Usar only() o except() para seleccionar varios**

`$datos = $request->only(['categorias', 'tipos']);`

`$datos = $request->except(['tipos']);` //devuelve todos los campos excepto los indicados

- **\$request->has('campo')**

Devuelve true si el campo existe en la petición (aunque esté vacío).

- **filled()**

Muy útil cuando necesitas comprobar si un campo fue enviado con algún valor (no vacío).

Devuelve true si:

- El campo existe en la solicitud
- Y tiene un valor distinto de null o cadena vacía ("")

`filled()` es mejor que `has()` si quieres evitar cadenas vacías.

- **\$file = \$request->file('campo')**
Para obtener un archivo subido. Puedes luego usar:
 - **\$file->getClientOriginalName()**
 - **\$file->getSize()**
 - **\$file->store('uploads','public')** → guarda el archivo en /storage/app/public/uploads, con un nombre aleatorio generado automáticamente que permite evitar colisión de nombres. Devuelve la ruta y el nombre aleatorio generado.
 - **\$file->storeAs('uploads', \$file->getClientOriginalName(), 'public')** → guarda el archivo y manteniendo el nombre original
- **\$request->boolean('campo')**
Convierte el valor del campo a true o false, útil para checkboxes
- **\$request->validate([...])**
Valida el formulario y devuelve los datos validados o lanza un error automáticamente.

```
// Validar y guardar producto
$validated = $request->validate([
    'name' => 'required|string|max:100',
    'description' => 'required|string',
    'price' => 'required|numeric|min:0',
    'category_id' => 'required|exists:categories,id',
    'image' => 'required|image|mimes:jpg,jpeg,png|max:2048'
]);

$product = new Product($validated);
```

Aquí, además, se crea un objeto producto nuevo asignándole, de golpe, todos los datos procedentes del formulario. Esto es posible siempre que el modelo Product tenga definidos en \$fillable todos los campos que están en \$validated.

Validaciones útiles en Laravel:

Regla	Qué valida
required	Campo obligatorio.
Nullable	Puede estar vacío.
String	Debe ser texto.
numeric	Debe ser un número.
Integer	Debe ser un número entero.
Email	Formato de correo electrónico válido.

Regla	Qué valida
min:n / max:n	Mínimo / máximo (para números o caracteres).
between:min,max	Valor o longitud dentro de un rango.
confirmed	Usado para validar campos que deben repetirse, como una contraseña. Laravel espera que haya otro campo con el mismo nombre + _confirmation. Por ejemplo, con un formulario con estos dos controles: <pre><input type="password" name="password" id="password" class="form-control" required></pre> <pre><input type="password" name="password_confirmation" id="password_confirmation" class="form-control" required></pre> En el controlador: <pre>\$request->validate(['password' => 'required min:8 confirmed']);</pre>
unique:tabla,columna	El valor no debe existir en esa tabla/columna.
exists:tabla,columna	El valor debe existir en esa tabla/columna.
date / before:fecha / after:fecha	Fecha válida y restricciones temporales.
in:val1,val2,...	El valor debe ser uno de los especificados.
Boolean	Acepta true, false, 1, 0.
Array	Debe ser un array.
file / image	Debe ser un archivo / imagen.
mimes:jpg,png	Tipos MIME permitidos.
size:valor	Tamaño exacto (número, archivo o string según contexto).

Ejercicio: Incorpora estas validaciones en el servidor para los datos de nuevos EMPLE y DEPART, así como para los formularios de los updates de registros existentes.

Ejercicio2: En los formularios de nuevos emplead@s, pon un campo nuevo para subir la foto de cada nuevo emplead@, y almacena la foto en disco, mientras que en la Base de Datos almacena la ruta y el nombre del archivo.

Ten en cuenta que si la foto se almacena utilizando el método `$file->store('fotos','public')`, se están almacenando en `/storage/app/public/fotos`.

Para luego recuperar la foto y mostrarla, sería desde la vista:

```
@if (!empty($emple->foto))
    
@endif
```

Donde `{{asset('storage/' . $emple->foto)}}` buscará la foto en `/public/storage`.

Luego es necesario hacer un enlace simbólico entre las rutas `/storage/app/public/fotos` y `/public/storage` para que las búsquedas en `/public/storage` realmente busquen en `/storage/app/public/fotos`. Para ello ejecutamos el comando:

php artisan storage:link

MÁS SOBRE LOS ARCHIVOS CONTROLADORES

Para crear un controlador con los métodos básicos para un CRUD tradicional, ideal para controladores que devuelven vistas (HTML), como en aplicaciones web tradicionales (Blade, etc.).

php artisan make:controller NombreDelControlador --resource

Esto generará un controlador con los métodos básicos para manejar operaciones CRUD, como:

- `index()`: Mostrar una lista de recursos.
- `create()`: Mostrar un formulario para crear un recurso.
- `store()`: Almacenar un recurso recién creado.
- `show($id)`: Mostrar un recurso específico.
- `edit($id)`: Mostrar un formulario para editar un recurso.
- `update($id)`: Actualizar un recurso existente.
- `destroy($id)`: Eliminar un recurso.

```
php artisan make:controller CategoryController --resource
```

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class CategoryController extends Controller
{
    public function index() { /* Mostrar lista de categorías */ }
    public function create() { /* Mostrar formulario para crear un categoría */ }
    public function store(Request $request) { /* Guardar el categoría */ }
    public function show($id) { /* Mostrar un categoría específico */ }
    public function edit($id) { /* Mostrar formulario para editar un categoría */ }
    public function update(Request $request, $id) { /* Actualizar un categoría existente */ }
    public function destroy($id) { /* Eliminar un categoría */ }
}
```

Definir las rutas a los recursos:

Si utilizaste la opción `--resource`, puedes registrar **todas las rutas a la vez** de forma automática:

```
use App\Http\Controllers\NombreDelControlador;
```

```
Route::resource('nombre-recurso', NombreDelControlador::class);
```

Esto generará automáticamente las siguientes rutas:

Método HTTP	Ruta	Acción del controlador	Nombre de la ruta
GET	/products	index	products.index
GET	/products/create	create	products.create
POST	/products	store	products.store

Método HTTP	Ruta	Acción del controlador	Nombre de la ruta
GET	/products/{id}	show	products.show
GET	/products/{id}/edit	edit	products.edit
PUT/PATCH	/products/{id}	update	products.update
DELETE	/products/{id}	destroy	products.destroy

Explicación de cada método:

1. **index:** Responde a una petición GET /products. Es para listar productos.
2. **create:** Responde a una petición GET /products/create. Muestra un formulario para crear productos.
3. **store:** Responde a una petición POST /products. Es para guardar un nuevo producto en la base de datos (por ahora solo devuelve un mensaje).
4. **show:** Responde a una petición GET /products/{id}. Muestra un producto específico según el ID proporcionado.
5. **edit:** Responde a una petición GET /products/{id}/edit. Muestra un formulario para editar un producto existente.
6. **update:** Responde a una petición PUT o PATCH /products/{id}. Es para actualizar un producto en la base de datos.
7. **destroy:** Responde a una petición DELETE /products/{id}. Es para eliminar un producto.

El valor que uses debe representar la colección de recursos que administra el controlador. Algunas recomendaciones:

- **Plural y en minúscula:** Es una convención usar el plural, ya que el controlador de recursos administra una colección de elementos (ejemplo: productos, usuarios, pedidos).
- **Descriptivo:** Usa un nombre que refleje el tipo de datos o recursos que estás gestionando.

Ejemplo:

/routes/web.php

```
use App\Http\Controllers\CategoryController;
```

```
Route::resource('categories', CategoryController::class);
```

Ahora puedes verificar las rutas con el comando:

php artisan route:list

```
GET|HEAD products ..... products.index > productsController@index
POST products ..... products.store > productsController@store
GET|HEAD products/create ..... products.create > productsController@create
GET|HEAD products/{product} ..... products.show > productsController@show
PUT|PATCH products/{product} ..... products.update > productsController@update
DELETE products/{product} ..... products.destroy > productsController@destroy
GET|HEAD products/{product}/edit ..... products.edit > productsController@edit
```

Para probar de forma básica y rápida la aplicación y verificar que las rutas están funcionando correctamente, puedes empezar añadiendo respuestas simples en cada método del controlador. Estas respuestas pueden ser mensajes o datos básicos que se devuelven al navegador.

```
class ProductsController extends Controller
{
    // Mostrar una lista de productos
    public function index()
    {
        return "Aquí se mostrarán todos los productos (ruta GET /products)";
    }

    // Mostrar un formulario para crear un nuevo producto
    public function create()
    {
        return "Aquí se mostrará el formulario para crear un producto (ruta GET /products/create)";
    }
}
```

Inicia el servidor de desarrollo →

```
php artisan serve
```

Prueba cada ruta en tu navegador:



El resto de peticiones las realizamos con la extensión Rest Client de VSC:

```
Send Request
✓ DELETE http://localhost:8000/products/1 HTTP/1.1
Content-Type: application/json

###
Send Request
✓ POST http://localhost:8000/products HTTP/1.1
Content-Type: application/json
{
    "name": "Producto de prueba",
    "price": 100
}
###
Send Request
✓ PUT http://localhost:8000/products/1
```

Si da error de validación de token, se debe a un problema con la verificación CSRF (Cross-Site Request Forgery). Laravel protege todas las solicitudes web (/routes/web.php) con un middleware que incluye:

- Protección contra ataques CSRF
- Manejo de sesiones y cookies
- Pensado para aplicaciones web con formularios HTML (Blade, etc.)

Esto significa que si haces peticiones POST, PUT, DELETE a rutas en web.php desde un cliente como REST Client o Postman, Laravel te exige un token CSRF, y si no lo envías, te responde con error 419 (CSRF token mismatch).

Para solucionar este problema de la validación CSRF en métodos distintos de GET vamos a hacer como si creásemos una API REST, pero solo para probar los métodos de este modo.

Para meter TODOS los métodos como si fuera una API, hay varias opciones según sea la versión del proyecto Laravel:

Laravel 12

- Crea el archivo routes/api.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\DepartController;

Route::apiResource('departs', DepartController::class);
```

Comenta este mismo enrutamiento en el fichero web.php porque volveremos a poner el enrutamiento aquí.

- Modifica tu bootstrap/app.php para incluir también las rutas de API. Debe quedar así:

```
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__ . '/../routes/web.php',
        api: __DIR__ . '/../routes/api.php',
        commands: __DIR__ . '/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        //
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

- Limpia la caché:

```
php artisan route:clear
php artisan config:clear
php artisan cache:clear
composer dump-autoload
```

- Verifica las rutas:

```
GET|HEAD / .....
GET|HEAD api/departs .....
POST api/departs .....
GET|HEAD api/departs/{depart} ...
PUT|PATCH api/departs/{depart} ...
DELETE api/departs/{depart} ...
GET|HEAD storage/{path} .....
GET|HEAD up .....
```

Como ves, las rutas comienzan automáticamente con api/ seguido del recurso indicado en el router.

/routes/api.php está diseñado específicamente para APIs:

- No usa CSRF
- No maneja sesiones
- Pensado para recibir peticiones de clientes externos (apps, frontend, Postman, REST Client...)
- Devuelve respuestas JSON
- Soporta perfectamente los métodos GET, POST, PUT, DELETE, etc., sin tokens extra.

Como **Laravel 12** ya no trae api.php por defecto, tú debes:

- Crear el archivo routes/api.php
- Decirle a Laravel que lo cargue, añadiéndolo en bootstrap/app.php
- Definir ahí tus rutas de API, como los POST, PUT, DELETE

Pero si tu objetivo final no es crear una API, sino una aplicación web tradicional con vistas Blade y formularios HTML para hacer un CRUD, entonces debes usar las rutas en web.php.

Y en ese caso, sí debes aceptar que Laravel aplique CSRF, porque:

- Los formularios HTML requieren protección CSRF.
- Los métodos POST, PUT, DELETE funcionan dentro de formularios usando `_token` y `@csrf`.
- Las respuestas serán vistas HTML, no JSON.

Método del controlador	Método HTTP	Cómo se llama desde la vista (Blade)	Descripción
<code>index(){ obtiene todos los registros de la BD se los pasa a la vista }</code>	GET	<code></code>	Lista de recursos
<code>create(){ llama al formulario para crear }</code>	GET	<code></code>	Formulario para crear
<code>store(Request \$request){ almacena el registro validándolo redirecciona a index }</code>	POST	<code><form method="POST" action="{{ route('posts.store') }}"></code>	Guardar nuevo recurso
<code>show(\$post_id)</code>	GET	<code></code>	Ver un recurso específico
<code>edit(\$post_id){ Busca el registro Se lo pasa a la vista }</code>	GET	<code></code>	Formulario para editar
<code>update(Request \$request, \$post_id){ Busca el registro con ese id Lo actualiza en la BBDD validándolo }</code>	PUT / PATCH	<code><form method="POST"> @method('PUT')</code>	Actualizar recurso
<code>destroy(\$post_id){ Busca el registro Lo borra de la BBDD Redirecciona a index }</code>	DELETE	<code><form method="POST"> @method('DELETE')</code>	Eliminar recurso

HTML solo soporta los métodos GET y POST de manera nativa. Para usar PUT, PATCH, o DELETE, Laravel usa un campo oculto llamado `_method`, y en Blade se simplifica así:

```
<form action="{{ route('posts.update', $post) }}" method="POST">
  @csrf
  @method('PUT')
  <!-- Campos del formulario -->
  <button type="submit">Actualizar</button>
</form>
```

Y para eliminar:


```
<form action="{{ route('posts.destroy', $post) }}" method="POST">
    @csrf
    @method('DELETE')
    <button type="submit">Eliminar</button>
</form>
```

10. CREACIÓN DE UNA API REST

1. Crea un nuevo proyecto en Laravel

```
composer create-project laravel/laravel <nombre_ejemplo>
```

O bien con Laravel:

Laravel new nombreProyecto

2. Configura la base de datos en el fichero .env y ejecutar las migraciones

php artisan migrate

3. Crea y edita el modelo y la migración para la tabla sobre la queramos realizar la API

```
php artisan make:model Book -m
```

Editar migración de la tabla books:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('books', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
            $table->string('title');
            $table->string('author');
            $table->integer('published_year');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('books');
    }
};
```

Ejecutar la migración:

```
> php artisan migrate
```

4. Define el modelo Book

Edita el archivo app/Models/Book.php:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

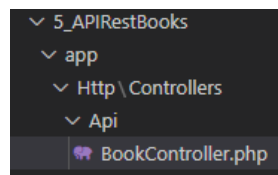
class Book extends Model
{
    use HasFactory;

    protected $fillable = ['title', 'author', 'published_year'];
}
```

5. Crea el controlador API

```
php artisan make:controller Api/BookController --api
```

Crea el controlador en:



Genera automáticamente los métodos:

```
public function index()
```

```
public function store(Request $request)
```

```
public function show(string $id)
```

```
public function update(Request $request, string $id)
```

```
public function destroy(string $id)
```

6. Define las rutas

Abre (si no existe, créalo) routes/api.php y agrega:

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\BookController;

Route::apiResource('books', BookController::class);
```

Route::apiResource es una instrucción de Laravel que genera automáticamente todas las rutas típicas de una API RESTful para un recurso:

Método HTTP	Ruta	Acción	Método del controlador
GET	/books	index	lista todos los libros
POST	/books	store	crea un nuevo libro
GET	/books/{id}	show	muestra un libro
PUT/PATCH	/books/{id}	update	actualiza un libro
DELETE	/books/{id}	destroy	elimina un libro

apiResource no crea las rutas create y edit porque esas vistas se usan en proyectos web, no en API.

Ya hemos visto anteriormente que Route:resource crea 7 métodos e incluye rutas con vistas (create, edit), mientras que Route:apiResource solo crea 5 métodos, y en una API no hay formularios ni vistas.

No utilices apiResource si no quieres implementar una API con CRUD completo (index, store, show, update, destroy) o bien las acciones no son CRUD (login, logout, search, profile,...) .

Incorpora esta ruta en /Bootstrap/app.php:

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    );
```

7. Implementa los métodos en el controlador

```
public function index(){
    return response()->json(Book::all(), 200);
}
```

```
public function store(Request $request){
    $request->validate([
        'title' => 'required|string|max:255',
        'author' => 'required|string|max:255',
        'published_year' => 'required|integer|min:1000|max:9999'
    ]);

    $book = Book::create($request->all());
    return response()->json($book, 201);
}
```

```
public function show($id){
    $book = Book::findOrFail($id);
    return response()->json($book, 200);
}
```

```
public function update(Request $request, $id){
    $book = Book::findOrFail($id);

    $request->validate([
        'title' => 'sometimes|required|string|max:255',
        'author' => 'sometimes|required|string|max:255',
        'published_year' => 'sometimes|required|integer|min:1000|max:9999'
    ]);

    $book->update($request->all());
    return response()->json($book, 200);
}
```

```
public function destroy($id)
{
    Book::destroy($id);
    return response()->json(null, 204);
}
```

8. Probar las rutas con RESTClient de VSC o Postman

Primero veremos si las rutas están bien definidas en la aplicación:

```
php artisan route:list
```

```
GET|HEAD / .....
GET|HEAD api/books ..... books.index > Api\BookController@index
POST api/books ..... books.store > Api\BookController@store
GET|HEAD api/books/{book} ..... books.show > Api\BookController@show
PUT|PATCH api/books/{book} ..... books.update > Api\BookController@update
DELETE api/books/{book} ... books.destroy > Api\BookController@destroy
GET|HEAD storage/{path} ..... storage.local
GET|HEAD up .....
```

Probamos:

```
GET http://127.0.0.1:8000/api/books
Accept: application/json
```

```
POST http://localhost:8000/api/books
Content-Type: application/json
Accept: application/json
```

```
{
  "title": "Clean Code",
  "author": "Robert C. Martin",
  "published_year": 2008
}
```

```
PUT http://127.0.0.1:8000/api/books/1
Content-Type: application/json
Accept: application/json
```

```
{
  "title": "The Pragmatic Programmer"
}
```

```
DELETE http://127.0.0.1:8000/api/books/2
Accept: application/json
```

11. LARAVEL BREEZE

Laravel Breeze es un starter kit de autenticación mínima para Laravel, que utiliza herramientas internas de Laravel, como:

- Laravel Fortify → aporta la lógica de autenticación
- Laravel Sanctum → trabaja con tokens / sesiones
- Controladores y rutas ya preparados

Existen varios tipos de Breeze, debido a que Laravel separa claramente backend y frontend. Laravel Breeze es el mismo backend con distintas capas de presentación:

- Breeze API, para API REST, usa una autenticación basada en tokens (Sanctum) y no tiene frontend.
- Breeze Blade, para aplicaciones web clásicas. Usa una autenticación basada en sesiones y cookies, y usa Blade como frontend.
- Breeze React, para aplicaciones SPA. Usa cookies para autenticación y frontend React
- Breeze Vue, para aplicaciones SPA. Usa cookies para autenticación y frontend Vue
- Breeze Livewire, para aplicaciones web interactivas, usa como sistema de autenticación sesiones y cookies, y como frontend usa Livewire

Todos usan Fortify por debajo, cambiando solo la capa de presentación.

12. AUTENTICACIÓN DE APIs CON SANCTUM

Sanctum es una opción ligera y sencilla para autenticación de APIs. Basada en tokens personales (Bearer Token) o cookies de sesión (para Single Page Application-SPA), donde el token se envía en la cabecera:

Authorization: Bearer TOKEN

Sanctum no es JWT puro, aunque se comporta parecido.

Breeze API incluye automáticamente Sanctum.

1. Instala Sanctum

```
composer require laravel/sanctum
```

2. Publica la configuración solo si se quiere personalizar Sanctum.

Aunque este paso creará la migración necesaria para el siguiente paso.

Publicar la configuración en Laravel es un proceso que permite copiar los archivos de configuración predeterminados de un paquete (como Sanctum) en el directorio config/ de tu aplicación. Esto te permite personalizar la configuración según tus necesidades.

Cuando instalas Sanctum Laravel trae una configuración predeterminada que se encuentra dentro del paquete de Sanctum. Sin embargo, esta configuración no está accesible directamente en tu aplicación. Para modificarla, necesitas publicarla en config/.

Para publicar la configuración de Sanctum crea un archivo en config/sanctum.php, ésto te permite personalizar opciones como:

- La configuración de tokens API.
- El middleware de autenticación.
- La expiración de los tokens.

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
```

Si necesitas que los tokens API expiren automáticamente, puedes definirlo en:

```
'expiration' => 60, // Los tokens expiran en 60 minutos
```

Este proceso crea también la migración del siguiente paso.

3. Ejecuta la migración

Sanctum necesita la tabla personal_access_token en la Base de Datos.

```
php artisan migrate
```

4. Crear el controlador AuthController.php

Si usas Breeze API ya existen controladores (AuthenticatedSessionController,...)
Si haces API manual, se necesita un nuevo controlador

php artisan make:controller AuthController

```
class AuthController extends Controller
{
    // Login de usuario
    public function login(Request $request)
    {
        $credentials = $request->validate([
            'email' => 'required|email',
            'password' => 'required'
        ]);

        if (!Auth::attempt($credentials)) {
            return response()->json(['message' => 'Unauthorized'], 401);
        }

        $token=$request->user()->createToken('auth_token')->plainTextToken;
        //tb valdría-> $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json(['access_token' => $token, 'token_type' => 'Bearer']);
    }

    // Obtener datos del usuario autenticado
    public function user(Request $request)
    {
        return response()->json($request->user());
    }

    // Logout (revoca el token actual)
    public function logout(Request $request)
    {
        $request->user()->tokens()->delete();
        return response()->json(['message' => 'Logged out']);
    }
}
```

Este controlador implementa autenticación por token usando Sanctum:

Método login:

- comprueba que llegan los campos requeridos, si no llegan, Laravel responde automáticamente con error 422
- Comprueba email y password en la BBDD (attempt). Si no coinciden lanza un 401-Unauthorized. Si coinciden, se inicia la autenticación.
- Se crea un token personal en personal_access_tokens
- Se devuelve el token personal al cliente, que lo deberá usar en la cabecera **Authorization: Bearer TOKEN** para realizar las acciones que precisen autenticación.

Método user:

- Devuelve los datos del usuario autenticado.
- Solo funciona si la ruta está protegida con middleware('auth:sanctum')
- Esta acción se utiliza para comprobar si el token es válido, obtener datos del usuario actual y mantener la sesión en cliente SPA (Single Page Application) o app móvil.

Método logout:

- Cierra la sesión revocando el token actual.

5. Proteger rutas con el middleware auth:sanctum (en /routes/api.php)

```
use App\Http\Controllers\AuthController;

Route::middleware(['auth:sanctum'])->group(function () {
    Route::apiResource('books', BookController::class);
});
```

6. Habilita el trait HasApiTokens en el modelo User.php

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    /** @use HasFactory<\Database\Factories\UserFactory> */
    use HasApiTokens, HasFactory, Notifiable;
}
```

Ahora, el usuario deberá autenticarse antes de acceder a la API.

7. Agrega rutas de autenticación en api.php (login, logout, user)

Modifica tu archivo routes/api.php para incluir login, logout y user info
Se crea un grupo de rutas protegido por sanctum:

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\BookController;
use Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful;
use App\Http\Controllers\AuthController;

//Route::apiResource('books', BookController::class);

// Ruta para autenticación (login)
Route::post('/login', [AuthController::class, 'login']);

// Grupo de rutas protegidas con Sanctum
Route::middleware(['auth:sanctum'])->group(function () {
    Route::apiResource('books', BookController::class);

    // Ruta para obtener el usuario autenticado
    Route::get('/user', [AuthController::class, 'user']);

    // Ruta para cerrar sesión (logout)
    Route::post('/logout', [AuthController::class, 'logout']);
});
```

O bien individualmente:

```
Route::post('/login', [AuthController::class, 'login']);
Route::post('/logout', [AuthController::class, 'logout'])->middleware('auth:sanctum');
Route::get('/user', [AuthController::class, 'user'])->middleware('auth:sanctum');
```

8. Autenticar con REST Client en VS Code o Postman

Vamos a crear un usuario manualmente a través de la consola de Laravel (tinker):

```
php artisan tinker
```

```
\App\Models\User::create([  
    'name' => 'Admin user',  
    'email' => 'admin@example.es',  
    'password' => bcrypt('password')  
]);
```

Probamos si existe:

```
$user = \App\Models\User::first();  
...
```

Obtenemos el token:

```
$token=$user->createToken('test_token')->plainTextToken;
```

Creamos el fichero de peticiones (RESTClient de VSC)

```
POST http://127.0.0.1:8000/api/login  
Content-Type: application/json  
  
{  
  "email": "admin@example.es",  
  "password": "password"  
}
```

Devolverá el token, que copiamos a la petición:

```
GET http://127.0.0.1:8000/api/books  
Authorization: Bearer 4|KPfdUREAyitz9Fz0j40gAtyrKIotMvTUotNurqJY2240ddb7  
Accept: application/json
```

Si no incorporamos esa cabecera:

```
DELETE http://127.0.0.1:8000/api/books/2  
Accept: application/json
```

Nos dará el siguiente error:

```
{  
  "message": "Unauthenticated."  
}
```

Podemos cerrar la sesión, aunque si hemos puesto caducidad, se cerrará sola:

```
POST http://127.0.0.1:8000/api/logout  
Authorization: Bearer 8|AGa1BDNZDvmHdWVLWKHpYBUd1U4RLeIawCwiCvX8c6b73516
```

13. INCLUIR BREEZE BLADE PARA AUTENTICACIÓN DE USUARIOS EN WEB TRADICIONAL

Laravel Breeze proporciona la estructura mínima y sencilla para manejar autenticación, además de una base limpia para comenzar una aplicación web.

Breeze instala:

- Registro (/register)
- Inicio de sesión (/login)
- Restablecimiento de contraseña
- Verificación de email
- Confirmación de contraseña
- Cierre de sesión

Todo funcionando desde el primer minuto

Incluye vistas, controladores, rutas preconfiguradas (/routes/auth.php) y middleware configurado.

Para instalar Laravel:

composer require laravel/breeze - --dev

Copia dentro de tu proyecto todos los ficheros de blade (plantillas, controladores, rutas, middleware, vistas Blade...):

php artisan breeze:install blade

Instala las dependencias de frontend definidas en package.json (Tailwind, Vite, JS, ...):

npm install

Ejecuta las migraciones y crea las tablas necesarias en la base de datos, incluyendo:

- users
- password_reset_tokens
- failed_jobs
- sessions (si usas sesiones en base de datos)

php artisan migrate

Ejercicio: instala breeze en la aplicación CRUD de EMPLE y DEPART.

Crea una vista home que incorpore partials y donde se muestre el menú:

CRUD_DEPART	login	register
-------------	-------	----------

Para usuarios públicos el menú permitirá acceder al CRUD de DEPART, aparte de loguearse y registrarse.

Para usuarios logueados el menú será el siguiente:

CRUD_DEPART	CRUD_EMPLE	logout (nombreUsuario)
-------------	------------	------------------------

14. APLICACIÓN PRIETO EATS

Creamos una nueva aplicación Laravel, y le instalamos el starter kit breeze para blade.

GESTOR DE BASES DE DATOS POSTGRESQL

Vamos a utilizar postgresql como gestor de bases de datos, podemos instalar un contenedor Docker con una imagen de PostgreSQL. Crea el usuario administrador, su contraseña y la base de datos al arrancar el contenedor por primera vez mediante variables de entorno, algo como:

```
docker run -d \
  --name nombre_contenedor \
  -p \dt
  -e POSTGRES_USER=nombre_usuario \
  -e POSTGRES_PASSWORD=password \
  -e POSTGRES_DB=nombre_base_datos \
  postgres:16
```

Una vez arrancado el contenedor PostgreSQL, puedes entrar en el cliente de línea de comandos de este gestor de Bases de Datos llamado psql, mediante:

```
docker exec -it nombre_contenedor psql -U nombre_usuario -d nombre_base_datos
```

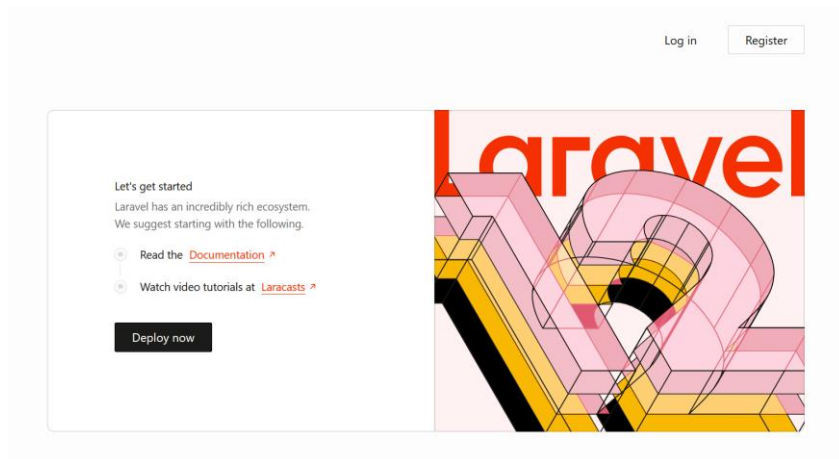
Esta línea de comandos nos va a permitir ejecutar consultas SQL y administrar bases de datos directamente desde la terminal. Podemos ver información del sistema mediante:

```
\l → listar bases de datos
\dt → listar tablas
\du → listar usuarios
\d+ nombre_tabla → muestra la estructura de la tabla
\c nombre_otra_bbdd → se conecta a otra base de datos indicada
\q → salir
```

Una vez ya tenemos el gestor de Base de Datos, modificamos el fichero .env:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=prieto_eats
DB_USERNAME=admin
DB_PASSWORD=admin123
```

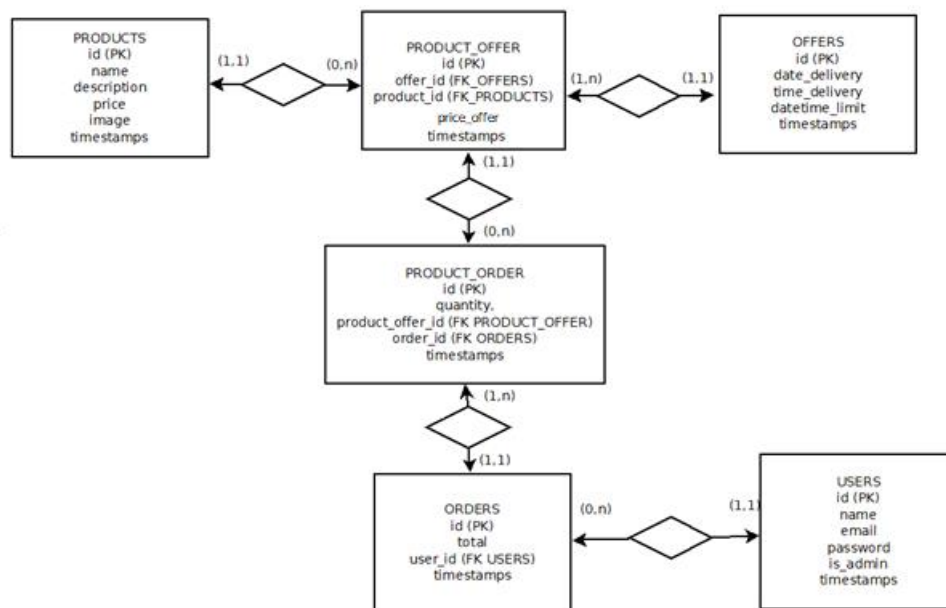
Hay que decirle a PHP que utilice el driver de PostgreSQL, para ello accedemos a php.ini y buscamos pgsql, nos dan dos líneas (extension=pdo_pgsql y extension=pgsql), las descomentamos y reiniciamos la consola o Apache, si estamos con XAMPP.



Ya podemos empezar a crear la parte del MODELO → migraciones, modelos,

MODELOS Y MIGRACIONES

Tendremos esta estructura de la Base de Datos:



Realiza, por tanto, las **migraciones** de TODAS las tablas, y los **modelos** de aquellas tablas fundamentales, ya que podríamos omitir las tablas intermedias de relaciones N:M que no aporten información adicional. En nuestro caso nos conviene realizar todas las tablas, incluso las intermedias, ya que representan entidades importantes para nuestra aplicación:

- **producto_offers** → representa a los productos ofertados, que se corresponden con productos reales, y que podrían tener información añadida como el precio de dicho producto ofertado, o stock de dicho producto ofertado, ésta opción última no la incluimos, pero sí el precio.
- **producto_orders** → representa los productos que se han pedido por los clientes y que se corresponden con productos incluidos en alguna oferta.

Tablas físicas (migraciones):

- **products**

Almacenará todos los productos (platos) disponibles a la venta.

Se almacenará la ruta y nombre del fichero de la imagen (\storage\app\public) del producto.

- **offers**

Representará las ofertas realizadas por el Departamento de Hostelería, cada oferta englobará a un conjunto de productos (platos)

- **product_offers**

Se refiere a los productos ofertados o incluidos dentro de cada oferta. Pondremos como PK el campo id, pero, para evitar que haya productos duplicados en la misma oferta, también pondremos los campos de las tablas PRODUCTS y OFFERS como valores UNIQUE (el conjunto de los dos).

Ponemos también el precio del producto en la oferta para permitir que el mismo producto en otra oferta pueda tener otro precio distinto.

- **orders**

Almacenará los pedidos confirmados por los usuarios

- **product_orders**

Almacenará los productos comprados en los pedidos, y que se refieren a los productos ofertados. Incluye la cantidad comprada de ese producto ofertado.

Las migraciones de estas tablas están en el aula virtual

Modelos en la aplicación:

- **Product**

```
class Product extends Model
{
    protected $fillable = [
        'name',
        'description',
        'price',
        'image'
    ];

    public function productsOffer()
    {
        return $this->hasMany(ProductOffer::class);
    }
}
```


- Offer

```
class Offer extends Model
{
    protected $fillable = [
        'date_delivery',
        'time_delivery',
        'datetime_limit'
    ];

    protected $casts = [
        'date_delivery' => 'date',
    ];

    public function productsOffer()
    {
        return $this->hasMany(ProductOffer::class);
    }
}
```

- ProductOffer

```
class ProductOffer extends Model
{
    protected $fillable = [
        'offer_id',
        'product_id',
        'price'
    ];

    public function offer()
    {
        return $this->belongsTo(Offer::class);
    }

    public function product(){
        return $this->belongsTo(Product::class);
    }

    public function productsOrder(){
        return $this->hasMany(ProductOrder::class);
    }
}
```

- ProductOrder
- Order

Los modelos de estas tablas están en el aula virtual.

Al tener así las tablas podemos utilizar la potencia de manejo de Eloquent para acceder a los datos:

Offer::with('productsOffer.product')->get() → selecciona los datos de las ofertas y sus productos ofertados, así como los de los productos asociados.

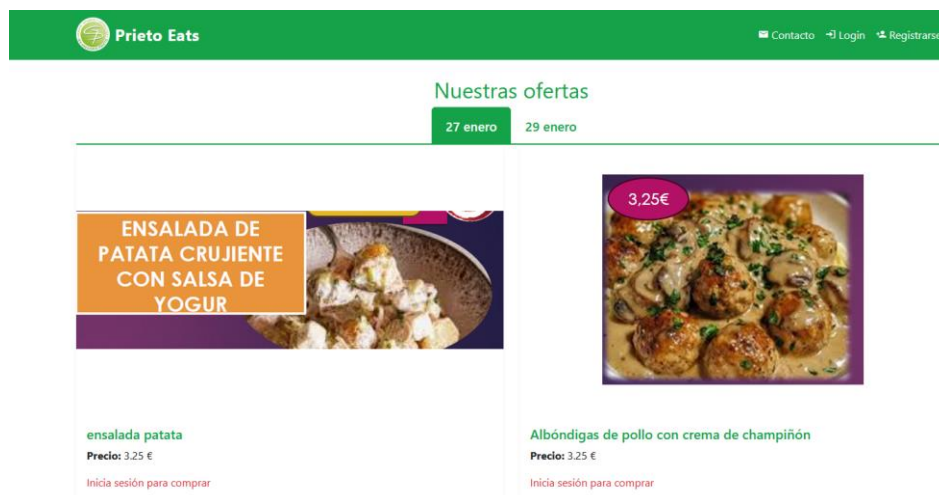
VISTAS INICIALES

Crea la estructura de partials (header, footer, content), la plantilla (layout) que las integre y aplícalo a la página home, así como a las páginas de login y register.

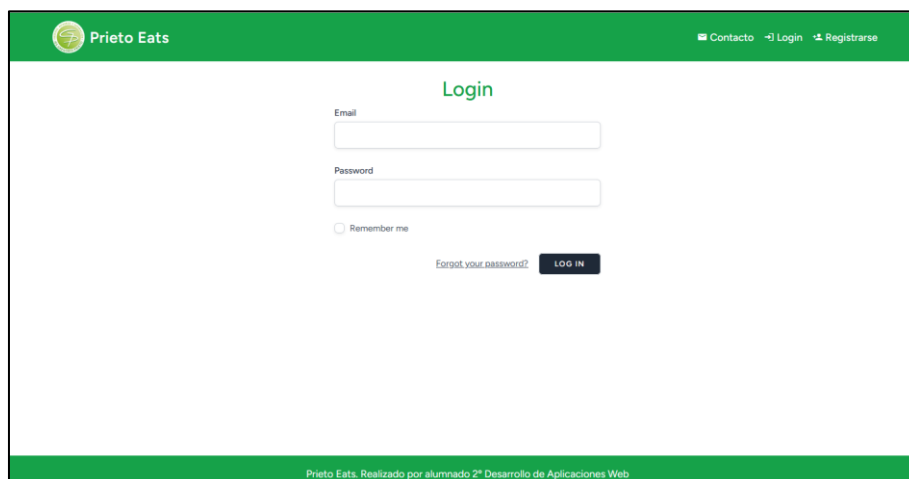
A continuación, vamos a crear el **home** de la aplicación, donde cualquier usuari@ podrá ver las ofertas existentes (que no hayan caducado), así como los productos ofertados dentro de dichas ofertas. Cada producto se mostrará en un card. La compra de los productos no estará disponible hasta que el usuari@ se haya logueado en el sistema.

En la página Home, realiza una paginación de 4 productos por página, aunque normalmente no habrá tantos productos ofertados.

Home



Página de login



Página de registro de nuevo usuario/a

Home cuando se ha hecho login

Como ves, una vez logueado, la misma página home muestra los productos, pero ahora sí que se muestra la opción de “Añadir al carrito”. Luego en la vista tienes que controlar si está logueado el usuario/a y mostrar un mensaje u otro:

```
@auth
    <form action="{{ route('cart.add', $menu->id) }}" method="POST">
        @csrf
        <button type="submit" class="btn btn-success">
            Añadir al carrito
        </button>
    </form>
@else
    <p class="text-danger">Inicia sesión para comprar</p>
@endauth
```

15. GESTIÓN DE SESIONES

Una sesión es un mecanismo que permite almacenar información del usuario entre diferentes solicitudes HTTP. Como HTTP es un protocolo sin estado, esto significa que cada vez que un usuario hace una solicitud a un servidor, éste no recuerda lo que ese usuario hizo antes. Las sesiones solucionan este problema al permitir almacenar datos que persisten durante la sesión del usuario.

Dónde se almacenan las sesiones en Laravel

Laravel maneja las sesiones utilizando diferentes drivers de almacenamiento, los cuales se pueden configurar en el archivo de configuración: **config/session.php**

Algunas opciones que Laravel ofrece para almacenar sesiones son:

Driver	Descripción
file	Archivos en <code>storage/framework/sessions</code> (por defecto).
cookie	Almacena la sesión en una cookie cifrada.
database	Usa una tabla en la base de datos.
redis	Almacena sesiones en memoria para mayor rapidez.
array	Almacena sesiones en un array (solo útil para pruebas).

Para cambiar el driver de sesión, edita el archivo `config/session.php`:

```
'driver' => env('SESSION_DRIVER', 'database'),
```

Si no está creada la tabla `session`, se crearía mediante:

```
php artisan session:table
```

```
php artisan migrate
```

Usos principales de las sesiones en una aplicación web

- **Autenticación de usuarios**

Cuando un usuario inicia sesión, sus datos (por ejemplo, su ID o nombre) se guardan en la sesión para identificarlo en cada solicitud sin necesidad de volver a ingresar sus credenciales.

- **Carritos de compra**

Las sesiones permiten almacenar los productos que un usuario ha agregado al carrito antes de completar una compra.

Esto permite que el usuario pueda seguir navegando sin perder los productos que seleccionó.

- **Almacenamiento de preferencias del usuario**

Se pueden guardar configuraciones temporales, como el idioma seleccionado, el modo oscuro, filtros de búsqueda, etc.

- **Protección contra ataques CSRF**

Laravel usa sesiones para almacenar tokens CSRF y evitar ataques donde un atacante intenta hacer solicitudes en nombre del usuario.

```
<input type="hidden" name="_token" value="{{ csrf_token() }}">
```

Laravel verifica este token en cada solicitud para asegurarse de que sea legítima.

- **Flash Messages (Mensajes Temporales)**

Sirven para mostrar mensajes de éxito o error después de una acción, como cuando un usuario envía un formulario.

```
session()->flash('mensaje', 'Registro exitoso!');
```

En la vista:

```
@if (session('mensaje'))  
    <div class="alert alert-success">{{ session('mensaje') }}</div>  
@endif
```

Estos mensajes desaparecen después de la siguiente solicitud.

- **Control de sesión y tiempo de expiración**

Las sesiones pueden expirar después de un tiempo de inactividad para mejorar la seguridad.

Si un usuario está inactivo durante un periodo prolongado, la sesión puede expirar, lo que requiere que el usuario se autentique nuevamente. Esto ayuda a prevenir accesos no autorizados en caso de que el usuario deje su dispositivo sin supervisión.

La configuración de la duración de la sesión se maneja a través de la variable `SESSION_LIFETIME` en el archivo `.env`:

```
SESSION_LIFETIME=120 // Duración de la sesión en minutos (120 minutos = 2 horas)
```

y el archivo de configuración `config/session.php`.

```
'lifetime' => (int) env('SESSION_LIFETIME', 120),
```

En este caso, el valor predeterminado es de 120 minutos si no se define otra cosa en el archivo `.env`.

Relación con `expire_on_close`:

Si tienes esta configuración en `config/session.php`:

```
'expire_on_close' => false, // No expira inmediatamente cuando el navegador se cierra
```

Y luego en el archivo `.env`:

```
SESSION_DRIVER=cookie // Si usas 'cookie' como controlador de sesión
```

Entonces la sesión permanecerá activa hasta que el tiempo de vida de la sesión (SESSION_LIFETIME) haya expirado, incluso si el navegador se cierra. Si quieres que expire cuando se cierre el navegador, pon:

```
'expire_on_close' => env('SESSION_EXPIRE_ON_CLOSE', true),
```

'expire_on_close' => true, // La sesión se destruye cuando se cierra el navegador

Cómo usar sesiones en Laravel

Laravel proporciona diferentes formas de manipular sesiones:

Guardar datos en sesión	<code>session(['clave' => 'valor']);</code>
Obtener datos de la sesión	<code>\$valor = session('clave');</code>
Obtener valor de la 'clave' en la sesión Si 'clave' no existe, devuelve [] en lugar de null	<code>session()->get('clave', [])</code>
Obtener todos los datos de la sesión	<code>\$datos = session()->all();</code>
Agregar un valor a un array almacenado en la sesión Cuando almacenas múltiples elementos bajo una misma clave	<code>session()->push('carrito', ['producto_id' => 1, 'cantidad' => 2]);</code> - Busca la clave en la sesión. - Si la clave ya tiene un array, agrega el nuevo valor al final del array. - Si la clave no existe, la crea con un nuevo array que contiene el valor agregado.
Sobreescribe el valor existente con esa clave	<code>session()->put('clave', \$valor)</code>
Verificar si una clave está en la sesión	<pre>if (session()->has('clave')) { // La clave existe en la sesión }</pre>
Verificar si una clave está definida, devuelve true aunque sea null	<pre>if (session()->exists('clave')) { // La clave existe en la sesión aunque sea nula }</pre>
Verifica si una clave existe en la sesión, pero además revisa si el valor no es null	<code>isset(session()->get('clave'))</code>
Eliminar una clave de la sesión	<code>session()->forget('clave');</code>
Eliminar varias claves de la sesión	<code>session()->forget(['clave1', 'clave2']);</code>
Limpiar toda la sesión	<code>session()->flush();</code>
Obtener y eliminar un valor de la sesión (Pull)	<code>\$valor = session()->pull('clave', 'valor_por_defecto');</code>
Usar mensajes flash (datos temporales)	A veces necesitas almacenar datos solo para la siguiente petición, como un mensaje de éxito después de un formulario: <pre>session()->flash('mensaje', 'Registro exitoso!');</pre> Luego puedes recuperar el mensaje en la vista: <pre>@if (session('mensaje')) <div class="alert alert-success"> {{ session('mensaje') }} </div> @endif</pre>

El ID de sesión permite a Laravel saber qué sesión pertenece a qué usuario. Cuando un usuario accede a una aplicación, su ID de sesión se almacena en una cookie (laravel_session), y en cada solicitud posterior, Laravel lo usa para cargar los datos de sesión correspondientes.

1. El usuario visita la página por primera vez

- Laravel genera un nuevo ID de sesión.
- Este ID se almacena en una cookie del navegador llamada laravel_session
- Laravel guarda información de sesión en el almacenamiento configurado (file, database, redis, etc.).

2. El usuario navega por la aplicación

- En cada solicitud, su navegador envía la cookie con el ID de sesión.
- Laravel usa este ID para recuperar los datos de sesión asociados.

3. Si el usuario cierra sesión o la sesión expira

- Laravel elimina la sesión y, si el usuario vuelve, se le asigna un nuevo ID de sesión.

Es buena práctica de seguridad regenerar el ID de sesión (session()->regenerate();) después del inicio de sesión, para evitar ataques de fijación de sesión (Session Fixation Attack). Este es un tipo de ataque en el que un atacante fuerza a un usuario a usar un ID de sesión previamente conocido. Si el usuario inicia sesión con ese ID de sesión, el atacante podría robar su cuenta simplemente reutilizando el mismo ID. Regenerando el ID de sesión, Laravel crea un nuevo ID de sesión pero mantiene los datos de sesión.

16. GESTIÓN DEL CARRITO DE LA COMPRA

La gestión de un carrito de compras se puede realizar de varias maneras, cada una con distintos niveles de complejidad, persistencia y escalabilidad.

- **Carrito basado en sesiones**

Los productos seleccionados se guardan en `$_SESSION` (en PHP puro) o en `session()` (en Laravel).

Ventajas:

- Fácil de implementar.
- No necesita base de datos.
- Ideal para carritos temporales.

Desventajas:

- Se pierde si el usuario borra cookies o la sesión expira.
- No funciona bien si el usuario cambia de dispositivo/navegador.

- **Carrito persistente en base de datos**

El carrito se guarda en una tabla `shopping_carts` con relación al usuario autenticado.

Ventajas:

- Persistente: el usuario puede recuperar su carrito en otra sesión o dispositivo.
- Escalable y profesional.
- Puedes añadir fecha, cantidad, estado, etc.

Desventajas:

- Requiere autenticación.
- Mayor complejidad de implementación.

Modelo típico:

- `users`
- `shopping_carts` (opcional, si usas `cart_items`)
- `cart_items`: `id`, `user_id`, `product_id`, `quantity`, `created_at`

- **Carrito híbrido (sesión + base de datos)**

Los productos se almacenan inicialmente en la sesión, y al iniciar sesión se sincronizan con la base de datos.

Ventajas:

- Lo mejor de ambos mundos: usabilidad + persistencia.
- Permite a usuarios anónimos usar carrito y recuperarlo al iniciar sesión.

- **Uso de paquetes externos**

Por ejemplo, Laravel tiene paquetes como:

- `bumbummen99/shoppingcart`: popular para Laravel, basado en sesión.
- O puedes integrar soluciones como Stripe, Snipcart o Shopify si quieres algo más completo.

Uso de la sesión para el carrito (basado en sesiones, sin persistencia)

Si tu aplicación no necesita persistir el carrito en una base de datos, puedes almacenarlo en la sesión del usuario.

Vamos a implementar un carrito mediante un array asociativo, donde la clave es el ID del producto, y su valor es un array con detalles del producto.

```
$cart = [
    101 => [
        'name' => 'menú 2 dic 2025',
        'price' => 8.00,
        'quantity' => 2,
        'image' => 'ruta/nombreImagen'
    ],
    202 => [
        'name' => 'menú 25 nov 2025',
        'price' => 8.00,
        'quantity' => 1,
        'image' => 'ruta/nombreImagen'
    ],
];
```

Según vamos a construir nuestro carrito de la compra, para recorrerlo podemos utilizar el siguiente bucle:

@foreach (\$cart as \$key => \$item)

Donde:

- \$key → 101, 202, ... (la clave del array principal)
- \$item → el array con name, price, etc.

Recordemos cómo se manejan los arrays asociativos en PHP:

Para el siguiente ejemplo:

```
$producto = [
    'nombre' => 'Laptop HP',
    'precio' => 750.00,
    'stock' => 10,
];
```

Acceder a valores de un array asociativo	<code>echo \$producto['nombre'];</code>
Modificar un valor en un array asociativo	<code>\$producto['precio'] = 800.00;</code>
Agregar un nuevo elemento	<code>\$producto['marca'] = 'HP';</code>
Verificar si una clave existe	<code>if (array_key_exists('stock', \$producto)) { echo "El producto tiene stock." }</code>

Verificar si una clave existe y no es null	<code>if (isset(\$producto['stock'])) { echo "Stock disponible."; }</code>
Eliminar un elemento	<code>unset(\$producto['stock']);</code>
Recorrer un array asociativo	<code>foreach (\$producto as \$clave => \$valor) { echo "\$clave: \$valor\n"; }</code>
Obtener todas las claves de un array	<code>\$claves = array_keys(\$producto); print_r(\$claves); // ['nombre', 'precio', 'marca']</code>
Obtener todos los valores de un array	<code>\$valores = array_values(\$producto); print_r(\$valores); // ['Laptop HP', 800, 'HP']</code>
Comprobar si un valor está en el array	<code>if (in_array('HP', \$producto)) { echo "La marca HP está en el array."; }</code>
Contar elementos de un array	<code>echo count(\$producto); // 3</code>
Unir arrays asociativos	<code>\$datos_extra = ['color' => 'Gris', 'garantia' => '2 años']; \$producto_completo = array_merge(\$producto, \$datos_extra);</code>
Obtener una parte de un array	<code>\$subset = array_slice(\$producto, 1, 2); print_r(\$subset); // ['precio' => 800, 'marca' => 'HP']</code>
Filtrar elementos en un array	<code>\$productos = ['Laptop' => 750, 'Mouse' => 25, 'Teclado' => 30,]; \$productos_caros = array_filter(\$productos, function (\$precio) { return \$precio > 50; }); print_r(\$productos_caros); // ['Laptop' => 750]</code>
Aplicar una función a todos los valores	<code>\$precios_con_iva = array_map(function (\$precio) { return \$precio * 1.21; }, \$productos);</code>
Ordenar por claves	<code>ksort(\$productos);</code>
Ordenar por valores	<code>asort(\$productos);</code>
Invertir un array	<code>\$productos_invertidos = array_reverse(\$productos, true);</code>

- Agregar botón "Añadir al carrito" en la vista de productos

```
@auth
<form action="{{ route('cart.add', $producto->id) }}" method="POST">
    @csrf
    <button type="submit" class="btn btn-success">Añadir al carrito</button>
</form>
@else
<p class="text-danger">Inicia sesión para comprar</p>
@endauth
```

- Crear las rutas para el carrito

```
Route::middleware('auth')->group(function () {
    Route::post('/cart/add/{id}', [CartController::class, 'add'])->name('cart.add'); // Agregar producto al carrito y añadir cantidad
    Route::get('/cart', [CartController::class, 'index'])->name('cart.index'); // Ver el carrito
    Route::delete('/cart/remove/{id}', [CartController::class, 'remove'])->name('cart.remove'); // Eliminar producto
    Route::delete('/cart', [CartController::class, 'clear'])->name('cart.clear'); // Vaciar carrito
});
```

...

- Crear el CartController

```
php artisan make:controller CartController
```

```
use App\Models\Product;
```

```
public function index()
{
    $cart = session()->get('cart', []);
    return view('cart.index', compact('cart'));
}
```

```
public function add(Request $request, $id){
    $product = Product::findOrFail($id);

    $cart = session()->get('cart', []);

    // Si el producto ya está en el carrito, incrementamos la cantidad
    if (isset($cart[$id])) {
        $cart[$id]['quantity']++;
    } else {
        // Agregar producto al carrito
        $cart[$id] = [
            'name' => $product->name,
            'price' => $product->price,
            'quantity' => 1,
            'image' => $product->image,
        ];
    }

    session()->put('cart', $cart);
    Log::info('Contenido del carrito:', $cart);
    return redirect()->route('cart.index')->with('success', 'Producto añadido al carrito.');
```

Log::info('mensaje') en Laravel es una forma de registrar un mensaje de log a nivel informativo en el archivo de logs de la aplicación (storage/logs/). Generalmente se utiliza para hacer un seguimiento de la ejecución de la aplicación. Por ejemplo, puedes usarlo para registrar información acerca de procesos que se completan correctamente, el estado de ciertos valores, o cualquier otro dato útil para la depuración o análisis.

```
public function remove($id){
    $cart = session()->get('cart', []);

    if (isset($cart[$id])) {
        unset($cart[$id]);
        session()->put('cart', $cart);
    }

    return redirect()->route('cart.index')->with('success', 'Producto eliminado del carrito.');
```

`unset($cart[$id])` → elimina una variable o un elemento específico de un array.

```
public function clear(){
    session()->forget('cart');

    return redirect()->route('cart.index')->with('success', 'Carrito vaciado.');
```

```
class CartController extends Controller
{
    public function index(){...
    }

    public function add(Request $request, $id){...
    }

    public function increase($id){...
    }

    public function decrease($id){...
    }

    public function remove($id){...
    }

    public function clear(){...
    }

    public function order(){...
    }
}
```

Las funciones `increase` y `decrease` son las que permiten aumentar o disminuir la cantidad de un determinado producto en el carrito de la compra.

La función `order` permitirá confirmar el pedido y que pase a la tabla **orders** y **order_items**. Validará si el carrito no está vacío, y en tal caso, calculará el total del pedido, creando un registro nuevo en la tabla **orders** y los correspondientes registros en la tabla **order_items**, y para que ambas operaciones se realicen conjuntamente, puedes definir dentro de una transacción (`DB::beginTransaction()`, `DB::commit()`, `DB::rollback()`). Elimina el carrito de la compra cuando se haya registrado el pedido.

- Crear la vista del carrito en resources/views/cart/index.blade.php

Quitamos toda referencia a componentes Laravel y utilizamos plantillas blade:

```
<!-- <x-app-layout>
<x-slot name="header">
    <h2 class="font-semibold text-xl text-gray-800 leading-tight">
        {{ __('Carrito de la compra')}}
    </h2>
</x-slot> -->

@extends('layouts.app')

@section('title', 'Inicio')

@section('content')
```

Imagen	Producto	Precio	Cantidad	Total	Acción
	menú 25 nov 2025	8.00 €	2	16.00 €	Eliminar
	pollo	12.00 €	1	12.00 €	Eliminar
				TOTAL PEDIDO = 28.00 €	
Vaciar Carrito		Seguir comprando		Confirmar pedido	

Prieto Eats. Realizado por alumnado 2º Desarrollo de Aplicaciones Web

Se ha de comprobar si el carrito está vacío y en este caso mostrar el mensaje:

Tu carrito está vacío.

Los botones los puedes realizar como formularios con método POST o DELETE, según sea la acción a realizar.

Ejercicio: implementa el carrito de la compra con sesiones, realizando además los métodos increase, decrease y order del cartController.php

Uso del carrito con persistencia en bases de datos

Persistir el carrito de la compra pasa por crear una tabla carts en la base de datos, donde se almacenarán los productos del carrito de la compra asociados al id del usuario logueado.

Crea la tabla del carrito con migraciones

```
php artisan make:migration create_cart_table
```

```
public function up(): void
{
    Schema::create('carts', function (Blueprint $table) {
        $table->id();
        $table->timestamps();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete(); // Relación con usuarios
        $table->foreignId('product_id')->constrained()->cascadeOnDelete(); // Relación con productos
        $table->integer('quantity')->default(1);
    });
}
```

- foreignId('user_id'): Crea una columna user_id en la tabla carts.
- constrained(): Laravel asume que user_id hace referencia a la columna id de la tabla users (por convención), y product_id a la tabla products. Donde en ambas tablas, los campos id deben ser PK.
- cascadeOnDelete(): Si un usuario se elimina, sus productos en el carrito también se eliminan.

Crea el modelo Cart

```
php artisan make:model Cart
```

```
class Cart extends Model{
    protected $fillable = ['user_id', 'product_id', 'quantity'];

    public function user() {
        return $this->belongsTo(User::class);
    }

    public function product() {
        return $this->belongsTo(Product::class);
    }
}
```

Actualiza el controlador del carrito

```
use Illuminate\Http\Request;
use App\Models\Cart;
use App\Models\Product;
use Illuminate\Support\Facades\Auth;
```

```
public function index()
{
    $cartItems = Cart::where('user_id', Auth::id())->with('product')->get();
    return view('cart.index', compact('cartItems'));
}
```

- `Cart::where('user_id', Auth::id())` filtra los registros de la tabla `carts`, obteniendo solo los elementos del usuario autenticado.
- `Auth::id()` obtiene el id del usuario actualmente autenticado.
- `with('product')` activa Eager Loading, es decir, carga también la información del producto relacionado en una sola consulta.

Equivaldría a la siguiente instrucción SQL:

```
SELECT * FROM carts
JOIN products ON carts.product_id = products.id
WHERE carts.user_id = 1;
```

Laravel busca un registro en la base de datos y devuelve una instancia del modelo `Cart`. El resultado de esta consulta es un objeto Eloquent, similar a:

```
$cartItem = (object) [
    'id'          => 12,
    'user_id'     => 5,
    'product_id' => 101,
    'quantity'   => 2,
    'created_at' => '2024-03-20 12:00:00',
    'updated_at' => '2024-03-20 12:05:00',
];
```

Como `$cartItem` es un objeto, accedes a sus atributos usando la sintaxis de objeto:

```
echo $cartItem->user_id; // 5
echo $cartItem->product_id; // 101
echo $cartItem->quantity; // 2
```

```
public function add(Request $request, $id){
    $product = Product::findOrFail($id);

    // Buscar si el producto ya está en el carrito del usuario
    $cartItem = Cart::where('user_id', Auth::id())->where('product_id', $id)->first();

    if ($cartItem) {
        // Si ya existe en el carrito, incrementar la cantidad
        $cartItem->quantity++;
        $cartItem->save();
    } else {
        // Si no existe, agregarlo al carrito
        Cart::create([
            'user_id' => Auth::id(),
            'product_id' => $id,
            'quantity' => 1
        ]);
    }

    return redirect()->route('cart.index')->with('success', 'Producto añadido al carrito.');
```

- `return redirect()->route('cart.index')->with('success', 'Producto añadido al carrito.');`
Redirecciona a la vista del carrito y envía un mensaje flash para notificarle que el producto se ha añadido correctamente.
- `route('cart.index')` genera la URL de la ruta nombrada `cart.index`.
- La ruta `cart.index` debería estar definida en `routes/web.php`
- `with('success', 'Producto añadido al carrito.')`
Este método almacena un mensaje flash en la sesión para que esté disponible en la siguiente solicitud.
 - `'success'` → Es la clave del mensaje flash.
 - `'Producto añadido al carrito.'` → Es el mensaje que se mostrará en la vista.

El mensaje puede mostrarse en la vista `cart.index` con:

```
@if(session('success'))
    <div class="alert alert-success">{{ session('success') }}</div>
@endif
```

Carrito de la compra

Imagen	Producto	Precio	Cantidad	Total	Acción
	Carne 4	\$15	2	\$30	Eliminar
	Pasta 2	\$15	1	\$15	Eliminar

[Vaciar Carrito](#)

En la tabla `cart` de la Base de Datos:

	id	created_at	updated_at	user_id	product_id	quantity
r	10	2025-06-12 17:33:57	2025-06-12 17:34:17	4	16	2
r	11	2025-06-12 17:34:46	2025-06-12 17:34:46	4	2	1

Método de la acción de borrar un producto del carrito (aunque tenga varias unidades):


```
public function remove($id){
    $cartItem = Cart::where('user_id', Auth::id())->where('product_id', $id)->first();

    if ($cartItem) {
        $cartItem->delete();
    }

    return redirect()->route('cart.index')->with('success', 'Producto eliminado del carrito.');
```

Método de limpiar el carrito de la compra:

```
public function clear(){
    Cart::where('user_id', Auth::id())->delete();
    return redirect()->route('cart.index')->with('success', 'Carrito vaciado.');
```

Cambios en la vista cart.index, ya que ahora no tenemos un array sino una colección de objetos Eloquent.

Cambiamos:

<code>\$item['name']</code>	<code>\$item->product->name</code>
<code>\$item['quantity']</code>	<code>\$item->quantity</code>

```
@foreach($cart as $item)
    <tr>
        <td></td>
        <td>{{ $item->product->name }}</td>
        <td>{{ $item->product->price }}</td>
        <td>{{ $item->quantity }}</td>
        <td>{{ $item->product->price * $item->quantity }}</td>
    </td>
```

```
<form action="{{ route('cart.remove', $item->product->id) }}" method="POST">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn btn-danger btn-sm">Eliminar</button>
</form>
```

```
<form action="{{ route('cart.clear') }}" method="POST">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn btn-warning">Vaciar Carrito</button>
</form>
```

17. GESTIÓN DE PERFILES/ROLES DE USUARIOS

En Laravel 12 no existe un sistema de roles “oficial” incluido en el core. La gestión de roles se construye encima del sistema de autenticación y autorización, y hay varios posibles enfoques:

Opción 1: Agregar una columna string "role" o boolean "is_admin" en la tabla users

A este campo “is_admin” se le da un valor predeterminado para la mayoría de los usuarios ('registered' si la columna es string o false si la columna es boolean)

Genera una nueva migración para agregar la columna:

```
php artisan make:migration role_admin_users --table=users
```

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('role')->default('registered');
    });
}
```

```
public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('role');
    });
}
```

Ejecuta la migración → `php artisan migrate`

- **Asignar un rol al registrarse**

Abre `app/Http/Controllers/Auth/RegisteredUserController.php` y modifica la función `store`, haciendo que cada nuevo usuario tome como valor predeterminado que “no es admin”:

```
$request->validate([
    'name' => ['required', 'string', 'max:255'],
    'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
    'password' => ['required', 'confirmed', Rules\Password::defaults()],
]);

$user = User::create([
    'name' => $request->name,
    'email' => $request->email,
    'password' => Hash::make($request->password),
    'role' => 'registered', // Asigna el rol por defecto
]);
```

Si quieres que ciertos usuarios sean administradores manualmente, puedes hacerlo a través de TablePlus o la consola Tinker de artisan:

php artisan tinker

```
> $user = App\Models\User::find(2);  
= App\Models\User {#6225  
    id: 2,  
    name: "admin",  
    email: "admin@iesgp.es",  
    email_verified_at: null,  
    #password: "$2y$12$Yo63jhRgAI4wmKb8gFSqo.At3/EVnjs4munRvRkEDKqkjZFeMtZim",  
    #remember_token: null,  
    created_at: "2025-03-10 20:31:43",  
    updated_at: "2025-03-10 20:31:43",  
    role: "registered",  
}
```

```
> $user->role="admin";  
= "admin"
```

```
> $user->save();  
= true
```

- Creamos un nuevo controlador para las acciones de los administradores
- Para proteger las rutas de los administradores, crearemos un middleware

Crea un middleware para restringir acceso a administradores:

```
php artisan make:middleware IsAdmin
```

Edita app/Http/Middleware/IsAdmin.php:

En la función handle indicamos que se seguirá solamente si está autenticado y es administrador:

```
public function handle(Request $request, Closure $next): Response  
{  
    if (auth()->check() && auth()->user()->role === 'admin') {  
        return $next($request);  
    }  
    abort(403, 'Acceso no autorizado');  
}
```

Dejaría pasar la ruta si cumple el if, y si no da el error 403.

Abre bootstrap/app.php y agrega manualmente la definición del middleware antes de devolver \$app:

Donde antes ponía:

```
->withMiddleware(function (Middleware $middleware) {
    //
})
```

Pon:

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'isAdmin' => \App\Http\Middleware\IsAdmin::class,
    ]);
})
```

- **Usar el Middleware en Rutas**

Una vez agregado, puedes proteger rutas en routes/web.php así:

```
Route::get('/admin/products/index', [AdminController::class, 'index'])
->middleware('auth', 'isAdmin')
->name('admin.index');
```

- Aplica los middlewares auth y isAdmin directamente a la ruta /admin/products/create.
- Cada vez que alguien acceda a esta ruta, ambos middlewares se ejecutarán.
- Si el usuario no está autenticado, el middleware auth lo redirige al login antes de verificar isAdmin.

O bien de esta otra forma:

```
Route::middleware('auth', 'isAdmin')->group(function () {
    Route::get('/admin/products/index', [AdminController::class, 'index'])->name('admin.index');
});
```

- Aplica auth e isAdmin a todo el grupo de rutas dentro del group(), lo que significa que todas las rutas dentro de este grupo solo estarán accesibles para usuarios autenticados y administradores.

También lo podemos poner así:

```
Route::middleware(['auth', 'isAdmin'])
->prefix('admin')
->name('admin.')
->group(function () {
    // Listado de productos con opciones para alta, baja y modificación
    Route::get('/products/index', [AdminController::class, 'index'])->name('products.index');
```

- prefix('admin') --> Hace que todas las rutas dentro de este grupo usen una URL que empieza por /admin --> /admin/products/index, ...
- name('admin.') --> Hace que todas las rutas dentro de este grupo tengan un nombre que empieza por admin. --> admin.products.index, ...

- **En las vistas**

Indicamos en Blade el contenido que solo los administradores deben ver:

1ª forma: comprobar el campo role en Blade

```
@auth
    @if (auth()->user()->role === 'admin')
        <!-- Contenido solo para administradores -->
    @endif
@endif
```

2ª forma: Crear un método isAdmin() en el modelo User:

```
public function isAdmin(){
    return $this->role === "admin";
}
```

En Blade:

```
@auth
    @if(auth()->user()->isAdmin())
        <!-- Contenido solo para administradores -->
    @endif
@endif
```

O bien

```
@auth
    @if(Auth::user()->isAdmin())
        <!-- Contenido solo para administradores -->
    @endif
@endif
```

3ª forma: crear una Directiva Blade personalizada (@admin)

En App\Providers\AppServiceProvider.php

```
use Illuminate\Support\Facades\Blade;

public function boot(): void
{
    Blade::if('admin', function () {
        return auth()->check() && auth()->user()->isAdmin();
    });
}
```

En la vista Blade:

```
@auth
    <!-- @if(Auth::user()->isAdmin()) -->
    @admin
        <!-- Menú solo para administradores -->
        <li class="nav-item dropdown">...
        </li>
    <!-- @endif -->
@endadmin
```

Opción limpia, profesional y cómoda si se usa en muchas vistas

Opción 2: Crear tablas roles y user_role

Entre las tablas users y roles habrá relación N – M, por eso creamos una tabla intermedia role_user.
En el modelo User.php de la tabla users:

```
public function roles(){
    return $this->belongsToMany(Role::class);
}
```

En el modelo Role.php:

```
public function users(){
    return $this->belongsToMany(User::class);
}
```

En las vistas Blade:

```
auth()->user()->roles->contains('name', 'admin');
```

Opción 3: Instalar la librería Spatie Laravel Permission

Se trata de un estándar profesional.

Consejo importante: Nunca confíes solo en Blade (@if) → Siempre protege rutas y controladores

18. PRIETO EATS. CREACIÓN DE NUEVA OFERTA CON CHECKBOX PARA ELEGIR PRODUCTOS

En la Vista

Tenemos un formulario para la creación de nuevas ofertas del tipo:

Alta de Oferta

Fecha recogida

dd / mm / aaaa

Horas recogida

Entre 13:35 y 14:30

Fecha y hora límite de pedidos

dd / mm / aaaa, -- : --

Listado de Productos

Selecciona producto en la oferta	Imagen	Nombre	Precio	Descripción
☐		ensalada patata	3.25 €	ensalada de patata con salsa de yogur
☐		Albóndigas de pollo con crema de champiñón	3.25 €	
☐		Flan parisien con haba tonka y vainilla	3.25 €	

Donde los checkbox para incluir los productos en la oferta son checkboxes del tipo:

```
<input type="checkbox" name="dish_selected[]" value="{{ $product->id }}">
```

Eso generará un array del tipo [idProducto1, idProducto2, ...] cuando se envíe al servidor con los id de los productos seleccionados dentro de la oferta.

En el Controlador

En el método del controlador que recibe los datos tendremos una primera validación de dichos datos:

```
$validated = $request->validate([
    'date_delivery' => 'required|date_format:Y-m-d|after:today',
    'time_delivery' => 'required|string|max:255',
    'datetime_limit' => 'nullable|date_format:Y-m-d\TH:i|after:now',
    'dish_selected' => 'required|array|min:1',
    'dish_selected.*' => 'integer|distinct|exists:products,id',
], [ /* mensajes personalizados */]);
```

Donde cada validación tiene el siguiente significado:

- `date_delivery` → obligatorio, formato YYYY-MM-DD, y debe ser posterior a hoy (after:today).
- `time_delivery` → obligatorio, texto corto (p. ej. "12:30–13:30").
- `datetime_limit` → opcional (nullable), pero si viene debe ser YYYY-MM-DDTHH:mm (típico de `<input type="datetime-local">`) y estar después de "ahora" (after:now).
- `dish_selected` → obligatorio, debe ser array con al menos 1 producto.
- `dish_selected.*` → cada elemento del array debe ser integer, sin duplicados (distinct) y existir en la tabla products (columna id).

Podemos acceder a los datos validados con `$validated['nombre_control']`, incluidos los ids de los productos seleccionados para la oferta que devolverá un array de dichos ids.

Tenemos que crear una oferta y varios productos ofertados, se puede realizar todo ello dentro de una transacción para que las dos operaciones vayan asociadas:

```
DB::transaction(function () use ($validated) {
    //crear el registro de la nueva oferta

    //obtener un array con los ids de los productos incluidos en la oferta
    $productIds = $validated['dish_selected'];

    //convertimos una lista de IDs de productos en una colección que contiene un array asociativo con los ids
    $rows = collect($productIds)->map(fn($id) => [
        'product_id' => $id
    ])->values()->all();

    //utilizamos estos datos para el método CreateMany que insertará los registros en producto_offers
    $offer->productsOffer()->createMany($rows);
}
```

Ponemos `use($validated)` para poder utilizar esa variable con los datos del formulario en la función.

Cómo se consigue crear una lista de arrays con los datos de los productos ofertados?

Supongamos que tenemos:

```
$productIds = [3, 7]; //procedente de los checkboxes de los productos
```

Y queremos obtener:

```
[
    ['product_id' => 3],
    ['product_id' => 7]
]
```

Para ello utilizamos este código:

```
$rows = collect($productIds)->map(fn($id) => [
    'product_id' => $id
])->values()->all();
```

Finalmente Podemos crear los registros de los productos ofertados mediante:

```
$offer->productsOffer()->createMany($rows);
```


19. PRIETO EATS. MODIFICACIÓN DEL CARRITO DE LA COMPRA PRIETO EATS A OFERTAS

Vamos a retocar el carrito de la compra, ya que ahora lo que el cliente selecciona son productos de una o de varias ofertas, luego es necesario organizar los productos solicitados por ofertas vigentes.

Si somos minimalistas en nuestro carrito y almacenamos únicamente lo necesario, podría quedar así:

```
cart = [  
  offer_id => [  
    productOffer_id => quantity  
  ]  
];
```

Ejemplo:

```
cart = [  
  10 => [ // offer_id = 10  
    101 => 2, // productOffer_id 101, cantidad 2  
    102 => 1  
  ],  
  12 => [ // offer_id = 12  
    150 => 1,  
    151 => 3  
  ]  
];
```

Mostrar el carrito de la compra

El método index del cart debe obtener toda la información de las ofertas, productOffer y productos para mandársela al carrito a mostrarse:

```
// cargamos de la sesión el carrito
$cart = session('cart', []); // [offer_id => [productOffer_id => qty]]

if (empty($cart)) {
    return view('cart.index', [
        'cart' => [],
        'offersById' => collect(),
        'productOffersById' => collect(),
    ]);
}

// conseguimos un array de ids de ofertas y productos-oferta, éste sin duplicados
$offerIds = array_keys($cart);

$productOfferIds = [];
foreach ($cart as $offerId => $items) {
    $productOfferIds = array_merge($productOfferIds, array_keys($items));
}
$productOfferIds = array_unique($productOfferIds);

// cargar datos de BD
$offersById = Offer::whereIn('id', $offerIds)
    ->get(['id', 'date_delivery', 'time_delivery'])
    ->keyBy('id');

$productOffersById = ProductOffer::with('product')
    ->whereIn('id', $productOfferIds)
    ->get(['id', 'offer_id', 'product_id'])
    ->keyBy('id');

// pasar todo a la vista
return view('cart.index', compact('cart', 'offersById', 'productOffersById'));
```

En la vista:

```
<div class="card mb-4 shadow-sm">
  <div class="card-body">
    @if (empty($cart))
      carrito está vacío.</p>
    @else
      @php
        $totalGeneral = 0;
      @endphp

      @foreach($cart as $offerId => $items)
        @php
          $offer = $offersById[$offerId] ?? null;
          $subtotal = 0;
        @endphp

        <div class="card mb-3">
          <div class="card-header">
            <strong>Oferta:</strong> {{ $offer->date_delivery->format('d/m/Y') }}
            <small class="text-muted">({{ $offer->time_delivery }})</small>
          </div>

          <div class="card-body">
            @foreach($items as $productOfferId => $qty)
              @php
                $po = $productOffersById[$productOfferId] ?? null;
                $prod = $po->product;
                $lineTotal = $prod->price * (int) $qty;
                $subtotal += $lineTotal;
                $totalGeneral += $lineTotal;
              @endphp
```

```
            @if ($po)
              <div class="row align-items-center py-2 border-bottom">
                <div class="col-3 col-md-3">
                  @if($prod->image)
                    name }}" class="img-fluid">
                  @endif
                </div>
                <div class="col-3 col-md-3">
                  <div class="fw-semibold">{{ $prod->name }}</div>
                  <div class="text-muted small">{{ $prod->description }}</div>
                </div>
                <div class="col-3 col-md-3">
                  <div class="fw-semibold">{{ number_format($prod->price,2) }} € <br></div>
                </div>
                <div class="col-3 col-md-3 mt-2 mt-md-2">
                  <i class="bi bi-dash"></i>
                  {{ $qty }}
                  <i class="bi bi-plus"></i>
                </div>
                <div class="col-3 col-md-3 text-end mt-2 mt-md-0">
                  <strong>{{ number_format($lineTotal,2) }} €</strong>
                </div>
              </div>
            @endif
          @endforeach
```

```
        </div>
        <div class="card-footer d-flex justify-content-between">
          <span>Subtotal oferta</span>
          <strong>{{ number_format($subtotal,2) }} €</strong>
        </div>
      </div>
    @endforeach
    <div class="text-end fs-5">
      <strong>Total:</strong> {{ number_format($totalGeneral, 2) }} €
    </div>
  @endif
</div>

{{-- TOTAL GENERAL --}}
<div class="card mt-4 shadow-sm">
  <div class="card-body d-flex justify-content-between align-items-center">
    <h4 class="mb-0">TOTAL GENERAL: {{ number_format($totalGeneral, 2) }} €</h4>
```



Programación Web en entorno servidor **LARAVEL**



20. VALIDACIÓN EN SERVIDOR Y EN CLIENTE DE FORMULARIOS

Por ejemplo, el formulario de creación de nuevos productos (solo administradores)

Formulario para crear un nuevo producto:

```
@extends('layouts.app')

@section('title', 'Alta de Productos')

@section('content')
<div class="container mt-4">
  <h2>Alta de Producto</h2>

  @if(session('success'))
    <div class="alert alert-success alert-dismissible fade show auto-dismiss">{{ session('success') }}</div>
  @endif

  <form action="{{ route('admin.products.store') }}" method="POST" enctype="multipart/form-data" class="needs-validation" novalidate>
    @csrf

    <div class="mb-3">
      <label for="name" class="form-label">Nombre</label>
      <input type="text" name="name" id="name" class="form-control @error('name') is-invalid @enderror" value="{{ old('name') }}" required>
      @error('name')
        <div class="invalid-feedback">
          {{ $message }}
        </div>
      @enderror
    </div>
  </form>
</div>
```

La expresión `value="{{ old('name') }}"` en Blade (Laravel) tiene el propósito de mantener el valor del campo del formulario después de un envío fallido (por ejemplo, cuando hay errores de validación).

`class="form-control @error('name') is-invalid @enderror"` se usa para añadir dinámicamente una clase CSS (`is-invalid`) a un campo de formulario si tiene errores de validación.

- `form-control`: clase de Bootstrap que da estilo al campo de formulario.
- `@error('name')`: directiva de Blade que comprueba si hay un error de validación asociado al campo 'name'.
- `is-invalid`: clase de Bootstrap que marca el campo con borde rojo y activa el mensaje de error si existe.
- `@enderror`: cierra la directiva `@error`.

Mensajes de error para campos inválidos (servidor):

Laravel mostrará errores si usas directivas como esta en cada campo:

```
@error('name')
  <div class="text-danger">{{ $message }}</div>
@enderror
```

Validación del lado del cliente (JavaScript):

Añade clases y feedback Bootstrap en cada campo:

```
<div class="mb-3">
  <label for="name" class="form-label">Nombre</label>
  <input type="text" name="name" id="name" class="form-control @error('name') is-invalid @enderror" value="{{ old('name') }}" required>
  @error('name')
    <div class="invalid-feedback">
      {{ $message }}
    </div>
  @enderror
</div>
```

Activa la validación en cliente con Bootstrap (HTML5 + JS)

Justo antes de cerrar el `</body>` o en tu plantilla base `app.blade.php`, añade:

```
<script>
// Bootstrap client-side validation
(() => {
    'use strict'
    const forms = document.querySelectorAll('.needs-validation')
    Array.from(forms).forEach(form => {
        form.addEventListener('submit', event => {
            if (!form.checkValidity()) {
                event.preventDefault()
                event.stopPropagation()
            }
            form.classList.add('was-validated')
        }, false)
    })
})()
```

Este script activa la validación del lado del cliente con estilos de Bootstrap, y mejora la experiencia del usuario mostrando retroalimentación visual (como bordes rojos o verdes) cuando envías un formulario.

```
((() => {
    'use strict'
```

Esta línea define una función anónima autoinvocada. 'use strict' activa el modo estricto de JavaScript para evitar errores silenciosos (por ejemplo, usar variables no declaradas).

```
const forms = document.querySelectorAll('.needs-validation')
```

Selecciona todos los formularios de la página que tengan la clase `.needs-validation`.

Para que funcione, tu formulario debe tener `<form class="needs-validation" novalidate>`

novalidate

desactiva la validación HTML5 por defecto, para que la controle el script.

```
Array.from(forms).forEach(form => {
    form.addEventListener('submit', event => {
```

Convierte la lista de formularios en un array y añade un listener a cada uno para el evento submit.

```
if (!form.checkValidity()) {
    event.preventDefault()
    event.stopPropagation()
}
```

Usa el método `checkValidity()` para comprobar si todos los campos del formulario son válidos.

Si hay algún campo inválido, se cancela el envío (`event.preventDefault()`).

```
form.classList.add('was-validated')
```

Esta clase activa los estilos visuales de Bootstrap en los campos:

is-valid para campos correctos

is-invalid para campos incorrectos

Bootstrap detecta esta clase para mostrar automáticamente los estilos en función del estado de cada input.

Y en tu formulario, añade la clase needs-validation y novalidate:

```
<form class="needs-validation" novalidate>
```

Cuando usas este tipo de validación con Bootstrap y el script:

```
<form class="needs-validation" novalidate>
```

Solo se valida automáticamente:

- required (campo obligatorio)
- type="email" (formato válido de correo)
- min, max, maxlength, pattern
- type="number" con step o min

Este script mejora la UX (experiencia de usuario) al:

- Impedir el envío inmediato del formulario si algún campo HTML5 está mal (por ejemplo, un required vacío).
- Mostrar los mensajes y bordes rojos (is-invalid) en tiempo real.
- Evitar una recarga innecesaria si el error ya es evidente en el navegador.



Programación Web en entorno servidor **LARAVEL**



Documentación:

- <https://www.paginaspersonales.unam.mx/app/webroot/files/4045/TemarioLaravel.pdf>